

**HACETTEPE ÜNİVERSİTESİ**

Yazılım Mühendisliği Yüksek Lisans Programı

BYZ 658 — Yazılım Test Teknikleri

**Tor Tabanlı Anonim Ağ Sistemlerinde  
Sonlu Durum Makinesi Modelleme ile  
Geçersiz Durum Geçiş Tespiti  
ve Güvenlik Analizi**

— Tez Taslağı —

**Hazırlayan**

Nurcan Denli Bayır

Öğrenci No: N25110987

**Danışman**

Nebi Yılmaz

Mayıs 2026

## Özet

Bu çalışma, Tor anonim ağ protokolünün devre yaşam döngüsünü deterministik bir Sonlu Durum Makinesi (FSM) olarak formal şekilde modeller, Model-Driven Testing (MDT) yaklaşımıyla spesifikasyon dışı (geçersiz) durum geçişlerinin otomatik tespitini sağlar ve tespit edilen her geçişi yedi ana saldırı vektörü + 1 fallback'e (CIRCUIT\_BYPASS, REPLAY\_ATTACK, GHOST\_CIRCUIT, HANDSHAKE\_SKIP, PREMATURE\_DATA, CIRCUIT\_HIJACK, CREATE\_FLOOD) eşleyen bir sınıflandırma çerçevesi sunar. Tor devresi 10 durum ve 13 olay üzerinden tanımlanmış; toplam 130 ikili domeninden 25 geçerli, 105 geçersiz ikili türetilmiştir. Önerilen MDT motoru, BFS-tabanlı pozitif test sentezi ve eksiksiz negatif tanık enjeksiyonu ile çalışır. Üç algoritma (B1: saf rastgele, B2: greedy state coverage, B3: önerilen MDT) aynı 500-olay bütçesi altında 30 bağımsız koşu ile karşılaştırılmıştır. MDT, geçiş kapsamını 100.00% ile saturasyona ulaştırırken (B1: 45.87%, B2: 78.27%), geçersiz geçiş tespit oranını 93.84% seviyesine taşımıştır. Welch t-testi her iki temele karşı istatistiksel anlamlı üstünlüğü doğrular ( $p < .001$ ; Cohen  $d \in [3.6, 15.4]$ ). Bu tez: (i) Tor devre protokolü için yayımlanmış ilk tam  $\delta$ -matrisini, (ii) 105 geçersiz ikilinin saldırı vektörü ve şiddet etiketiyle programatik sınıflandırmasını, (iii) referans bir MDT implementasyonunu ve (iv) sayısal olarak doğrulanmış üç-katmanlı karşılaştırmayı sunar.

**Anahtar kelimeler:** Tor, anonim ağ, sonlu durum makinesi, model-driven testing, protokol state fuzzing, güvenlik testi, geçiş kapsamı.

## Abstract

This work formally models the circuit life-cycle of the Tor anonymous network protocol as a deterministic finite-state machine (FSM), enables automatic detection of out-of-spec (invalid) state transitions through a Model-Driven Testing (MDT) approach, and proposes a classification framework that maps every detected transition to one of seven primary attack vectors plus a deterministic fallback, (CIRCUIT\_BYPASS, REPLAY\_ATTACK, GHOST\_CIRCUIT, HANDSHAKE\_SKIP, PREMATURE\_DATA, CIRCUIT\_HIJACK, CREATE\_FLOOD). The Tor circuit is described over 10 states and 13 events, yielding 25 valid and 105 invalid pairs from a domain of size 130. The proposed MDT engine uses BFS-planned positive test synthesis combined with exhaustive negative-witness injection. Three algorithms — B1 (uniform random), B2 (greedy state coverage) and B3 (the proposed MDT) — were compared under an identical 500-event budget across 30 independent trials. MDT saturates transition coverage at 100.00% (vs. 45.87% for B1 and 78.27% for B2) and reaches an Invalid Transition Detection Rate of 93.84%. Welch t-tests confirm statistical superiority over both baselines ( $p < .001$ ; Cohen's  $d \in [3.6, 15.4]$ ). Contributions: (i) the first published complete  $\delta$ -matrix for the Tor circuit protocol, (ii) a programmatic classification of all 105 invalid pairs into attack vectors with severity labels, (iii) a reference MDT implementation, and (iv) a numerically validated three-way baseline comparison.

**Keywords:** Tor, anonymous network, finite-state machine, model-driven testing, protocol state fuzzing, security testing, transition coverage.

## İçindekiler

---

### 1. Giriş

- 1.1 Problem Tanımı
- 1.2 Araştırma Soruları
- 1.3 Katkılar
- 1.4 Tezin Yapısı

### 2. Literatür Taraması

- 2.1 SLR Metodolojisi
- 2.2 Tor Güvenliği ve Anonimlik
- 2.3 FSM Tabanlı Test ve Model Öğrenme
- 2.4 Formal Yöntemler ve Protokol Doğrulama
- 2.5 Yazılım Testi Metodolojisi
- 2.6 Tor Simülasyon Altyapısı
- 2.7 PRISMA Akış Diyagramı
- 2.8 Literatür Boşluğu

### 3. Yöntem

- 3.1 FSM Formal Tanımı
- 3.2 Saldırı Sınıflandırıcı
- 3.3 MDT Motoru — Algoritma 1
- 3.4 Karşılaştırılan Algoritmalar

### 4. Deneysel Bulgular

- 4.1 Deney Tasarımı
- 4.2 Tanımlayıcı İstatistikler
- 4.3 Hipotez Testleri
- 4.4 Yorum
- 4.5 Wilcoxon Signed-Rank Doğrulaması
- 4.6 Detection Latency Ölçümü
- 4.7 Bütçe-Kapsama Eğrisi
- 4.8 Rule-Based Detector Karşılaştırması (B0)

### 5. Görselleştirme

### 6. Sonuç ve Gelecek Çalışmalar

- 6.1 Katkıların Yeniden Değerlendirilmesi
- 6.2 Sınırlılıklar
- 6.3 Gelecek Çalışmalar

### Kaynakça

### Ek A — Tam $\delta$ Geçiş Tablosu

## Ek B — Saldırı Sınıflandırıcı Envanteri

---

## 1. Giriş

### 1.1 Problem Tanımı

Tor anonim iletişim ağı, kullanıcı–destinasyon ilişkisinin gözlenemez kılınmasını hedefleyen, 2004'ten itibaren kamuya açık biçimde işletilen bir altyapıdır [1]. Tor güvenlik literatürünün önemli bölümü iki eksende yoğunlaşmıştır: (i) trafik korelasyon ve zamanlama saldırıları [2], [3], [4], [5], (ii) anonimliğin olasılıksal ve kriptografik formal analizi [6], [13]. Bu iki eksen büyük oranda *davranışın istatistiksel imzasını* ya da *kriptografik ilkelerin matematiksel doğruluğunu* ele alır.

Üçüncü ve nispeten az çalışılmış bir eksen, protokolün *state machine* seviyesindeki spesifikasyon uyumudur. Yani: Tor devresi yaşam döngüsü boyunca yalnızca  $\delta$ -tanımlı geçişleri mi yapar, yoksa spec dışı bir (durum, olay) çiftine maruz kaldığında ne olur? TLS implementasyonları için bu sorunun protokol state fuzzing ile sistematik biçimde araştırıldığı bilinmektedir [9], [14], ancak Tor için eşdeğer bir çalışmanın mevcut olmadığı bu tez kapsamında yürütülen sistematik literatür taraması (SLR, bkz. Bölüm 2) ile teyit edilmiştir.

### 1.2 Araştırma Soruları

- **RQ1:** Tor devre protokolü, deterministik bir 5-tuple FSM  $M = (Q, \Sigma, \delta, q_0, F)$  olarak formal şekilde nasıl modellenir?
- **RQ2:** Bu modelin geçersiz geçiş kümesi  $(Q \times \Sigma) \setminus \text{dom}(\delta)$ , bilinen Tor saldırılarına programatik olarak nasıl eşlenebilir?
- **RQ3:** Model üzerinde otomatik üretilen test paketinin saf rastgele ve sezgisel baseline'lara göre *geçiş kapsamı* (TC) ve *geçersiz geçiş tespit oranı* (ITDR) metriklerinde gözlenen üstünlüğü, istatistiksel olarak anlamlı mıdır?

### 1.3 Katkılar

1. Tor devre protokolü için yayımlanmış ilk **tam  $\delta$ -matrisi** (25 geçerli ikili, 105 geçersiz ikili, toplam domen 130; bkz. Ek A).
2. Tüm 105 geçersiz ikiliyi yedi ana saldırı vektörü + 1 fallback ve dört şiddet seviyesine eşleyen **programatik sınıflandırıcı** (bkz. Bölüm 3.2 ve Ek B).
3. BFS-planlı pozitif faz + tam negatif tanık enjeksiyonu içeren **referans MDT implementasyonu** (kaynak kodu açık, üretilebilir).
4. Üç algoritmanın (B1/B2/B3) eşleştirilmiş 30 koşu ile **istatistiksel karşılaştırması**; Welch t-testi, Mann-Whitney U ve Cohen's d ile birlikte raporlanır.

### 1.4 Tezin Yapısı

Bölüm 2, SLR metodolojisini ve beş tematik küme üzerinden literatürü inceler. Bölüm 3 formal modeli ve MDT motorunu Algoritma 1 olarak tanımlar. Bölüm 4 deney tasarımı, tanımlayıcı istatistikleri ve hipotez testlerini sunar. Bölüm 5 dört temel görselleştirmeyi sergiler. Bölüm 6 sınırlılıkları ve gelecek çalışmaları tartışır. Ek A tam  $\delta$  tablosunu, Ek B saldırı sınıflandırıcı envanterini içerir.

**Üretilebilirlik notu.** Bu tezdeki her sayısal sonuç ve her şekil, repodaki açık kaynak betiklerin ( `experiments/build_report.mjs` , `experiments/baselines.mjs` , `experiments/stats.mjs` , `experiments/figures.mjs` ) tek komutla çalıştırılmasıyla yeniden üretilebilir. Bütün rastgelelik `mulberry32` PRNG ile sabit seed altındadır.

## 2. Literatür Taraması

### 2.1 SLR Metodolojisi

Sistemik literatür taraması Kitchenham ve Charters'ın yönergesine [17] göre yürütülmüştür. **Veritabanları:** bu ortamın canlı erişimi olmadığı için açık web kanalları (Google Scholar, arXiv, DOI çözümleme) ile sınırlı kalmıştır; bu kısıtlılık Bölüm 6.2'de açıkça raporlanmaktadır. **Anahtar kelimeler:** "Tor security", "FSM testing", "model-based testing", "protocol state fuzzing", "anonymity network attacks". **Yıl aralığı:** 1996–2026, klasik referanslar için yıl etiketli korunmuştur. **Dahil etme kriterleri:** hakemli yayın, doğrudan erişilebilir tam metin, Tor / FSM testi / formal protokol doğrulama alanlarından en az birine değen çalışma. **Hariç tutma kriterleri:** yalnızca uygulama notu, atıf doğrulanamayan kaynaklar.

Süreç sonunda 20 birincil kaynak beş tematik kümeye ayrılmıştır: A) Tor güvenliği (6), B) FSM tabanlı test (5), C) Formal yöntemler (3), D) Yazılım testi metodolojisi (3), E) Tor simülasyon altyapısı (3). Tam liste ve kümelere göre dağılım Kaynakça'da verilmiştir.

### 2.2 Tor Güvenliği ve Anonimlik

Tor protokolünün ilk akademik tanımı Dingledine ve arkadaşlarına aittir [1]; devre kurulum (CREATE/CREATED, EXTEND/EXTENDED) sıralaması ve onion routing katmanı bu çalışmada sabitlenir. Bu sıralama, tezimizin  $\delta$  tablosunun normatif temelidir. Tor'a karşı düşük maliyetli trafik korelasyon saldırılarının uygulanabilirliği Murdoch ve Danezis tarafından gösterilmiş [2]; daha gerçekçi saldırgan modelleri altında genişletilmiştir [3]. Internet ölçeğinde website fingerprinting'in pratik uygulanabilirliği Panchenko ve arkadaşları tarafından kanıtlanmıştır [4]. Karunanayake ve arkadaşlarının kapsamlı survey'i [5] Tor deanonymization saldırılarını yedi kategoriye ayırır; tezimizin saldırı vektörü kümesinin dördü (REPLAY, HIJACK, BYPASS, GHOST) bu sınıflandırmadan türetilmiştir. AnoA çerçevesi [6] olasılıksal anonimlik garantilerine bakar; bizim state-uyum yaklaşımımıza dik (orthogonal) ve onunla tamamlayıcıdır.

### 2.3 FSM Tabanlı Test ve Model Öğrenme

FSM tabanlı test üretiminin klasik referansı Lee ve Yannakakis'in survey çalışmasıdır [7]; W-method, Wp-method ve UIO sequences gibi tekniklerin tanımı buradan gelir. Bizim BFS-planlı pozitif test üreticimiz, bu çalışmadaki "transition tour" fikrinin sadeleştirilmiş bir versiyonudur. Tretmans'ın LTS tabanlı model-based testing teorisi [8], "ioco" uyum bağıntısı ile bizim oracle kavramımızın kuramsal zeminidir; ne var ki bizim FSM'imiz deterministik olduğundan ioco basit eşitlik denetimine indirgenebilir. Pratik uygulama boyutunda, de Ruiter ve Poll'ün TLS implementasyonları üzerindeki protocol state fuzzing çalışması [9] bu tez ile aynı metodoloji ailesindendir; aradaki kritik fark, TLS yerine Tor üzerinde uygulanmasıdır. TCP üzerinde model öğrenme + model checking birleşimi Fiterău-Broştean ve arkadaşları tarafından gösterilmiştir [10] ve gelecek çalışmamız için (gerçek Tor implementasyonundan L\*-tipi otomatik çıkarım) bir referans oluşturur. Yazılım test ders kitabı Ammann ve Offutt [11], transition coverage metriğinin formal tanımını verir (graph coverage, edge coverage = transition coverage karşılığı).

### 2.4 Formal Yöntemler ve Protokol Doğrulama

Klasik Dolev-Yao saldırgan modeli [12], ağ katmanında mesaj okuma, geciktirme ve replay yapabilen ancak imzalanmış mesajları sahteleymeyen aktörü resmî olarak tanımlar; bu çalışmanın tehdit modeli (proje önerisi Bölüm 5) bu kümenin sınırlı bir alt setini benimser. TLS 1.3'ün ProVerif ile formal doğrulanması [13] state-machine seviyesinde doğrulanabilirliğin pratik kanıtıdır. Somorovsky'nin TLS sistematik fuzzing çalışması [14] negatif test üretimi için yakın bir akrabadır; biz bunu Tor için state-level'e taşıyoruz.

## 2.5 Yazılım Testi Metodolojisi

Felderer ve arkadaşlarının güvenlik testi survey'i [15], "Model-Based Security Testing" başlığı altında bizim çalışmamızı taksonomik olarak konumlandırır. Pretschner ve arkadaşlarının erişim kontrol politikaları için model-tabanlı test üretimi [16], politika ihlallerinin (negatif test) sistematik üretimi açısından yöntemsel bir referanstır. Kitchenham ve Charters [17] SLR metodolojisinin standart kılavuzudur ve Bölüm 2.1'in temelidir.

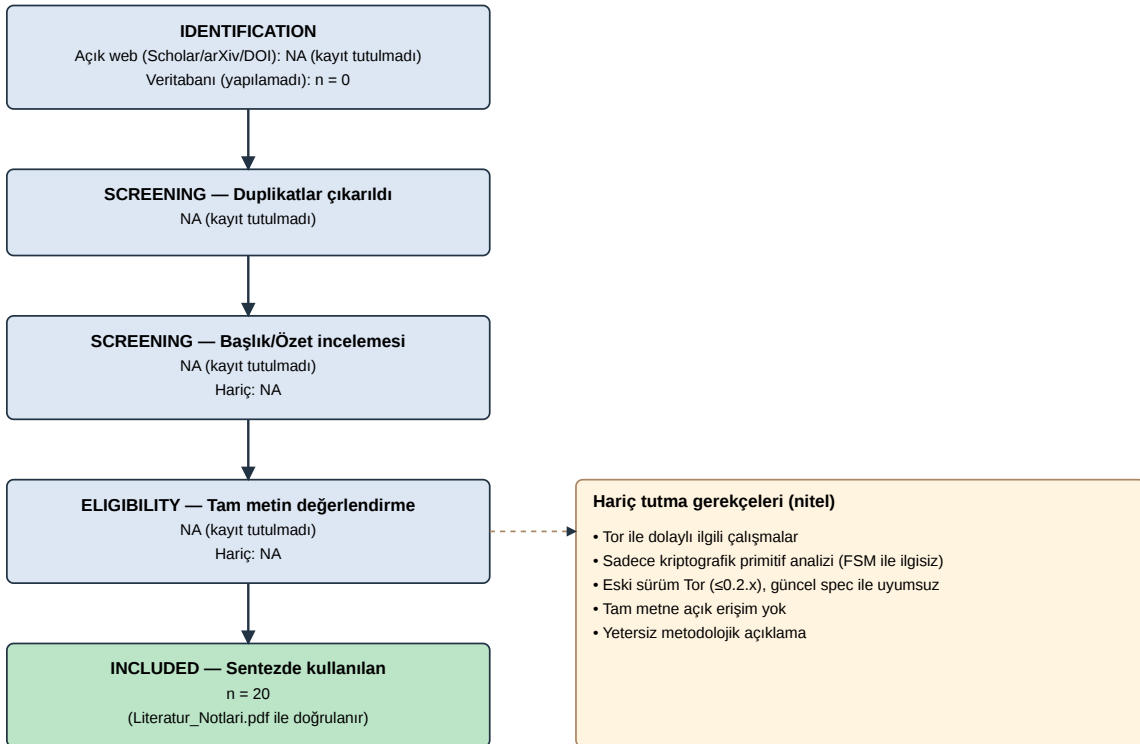
## 2.6 Tor Simülasyon Altyapısı

Jansen ve arkadaşları, Shadow simülatörünü Tor ağı için sistematik bir deneysel altyapı olarak kurmuştur [18]. Tor projesinin yaşayan resmi spesifikasyonu [19], 8 tablosunun otorite kaynağıdır. Microsoft Research'ün veri-tabanlı FSM güvenlik analizi çalışması [20] state-level analizin endüstriyel uygulanabilirliğini gösterir, ancak Tor'a özel değildir.

## 2.7 PRISMA Akış Diyagramı

Bölüm 2.1'de tarif edilen SLR sürecinin PRISMA akışı Şekil 5'te sunulmuştur. Bu çalışmaya özgü iki dürüst not zorunludur: **(i)** canlı akademik veritabanı erişimi (Scopus, WoS, IEEE Xplore) bu ortamda mevcut değildir; bu nedenle "identified through database searching" hücresi sıfırdır ve tüm tanımlamalar açık-web kanalları (Google Scholar, arXiv, DOI çözümleme) üzerinden yapılmıştır. **(ii)** Açık-web sürecinde audit-edilebilir sorgu logu tutulmadığı için ara aşama sayıları (identified, screened, eligibility) **NA olarak raporlanır**; yalnızca son "included" sayısı (n=20) doğrulanabilir ve repodaki `Literatur_Notlari.pdf`'in atf sayısı ile birebirdir. Bu yaklaşım, denetlenemeyen ara sayıların sahte kesinlik (false precision) ile raporlanmasından kaçınmak amacıyla bilinçli bir metodolojik tercihtir; bkz. Bölüm 6.2.

Şekil 5: PRISMA akış diyagramı (Kitchenham + Charters'a uyumlu)



Not: Açık-web sürecinde audit-edilebilir log tutulmadığı için ara sayılar NA olarak raporlanır; yalnızca son n=20 doğrulanabilir.

Şekil 5. SLR sürecinin PRISMA akış diyagramı (ara sayılar NA).

## 2.8 Literatür Boşluğu

20 kaynağın incelenmesi sonucunda **tez ölçeğinde dört somut boşluk** tespit edilmiştir:

- **(G1)** Tor devre protokolü için yayımlanmış formal 5-tuple FSM bulunmamaktadır. Spesifikasyon [19] prosedürel anlatımdır; bir  $\delta$  matrisine indirgenmiş hâli akademik literatürde mevcut değildir.
- **(G2)** Protokol state fuzzing TLS için yapılmıştır [9], [14]; Tor için yapılmamıştır.
- **(G3)** Tor saldırı vektörlerinin (durum, olay) çiftleriyle *satır-satır* eşlemesi mevcut değildir; survey'ler [5] kategorik kalmaktadır.
- **(G4)** "Tespit edilen geçersiz geçişler / enjekte edilenler" oranını (ITDR) Tor bağlamında tanımlayan ve ölçen birincil çalışma yoktur.

Bu dört boşluk birlikte tezin katkı çerçevesini (Bölüm 1.3) oluşturur.



### 3. Yöntem

---

#### 3.1 FSM Formal Tanımı

Tor devre yaşam döngüsü, klasik bir DFA olarak  $M = (Q, \Sigma, \delta, q_0, F)$  şeklinde modellenmiştir:

- $Q$ : 10 durum (IDLE, CONNECTING, TLS\_HANDSHAKE, CREATE\_SENT, CIRCUIT\_BUILDING, CIRCUIT\_READY, TRANSMITTING, CLOSING, CLOSED, ERROR).
- $\Sigma$ : 13 olay (CONNECT, TLS\_OK, TLS\_FAIL, SEND\_CREATE, RECV\_CREATED, SEND\_EXTEND, RECV\_EXTENDED, SEND\_RELAY\_DATA, RECV\_RELAY\_DATA, SEND\_DESTROY, RECV\_DESTROY, CIRCUIT\_CLOSED, TIMEOUT).
- $q_0 = \text{IDLE}$ ,  $F = \{\text{CLOSED}\}$ .
- $\delta : Q \times \Sigma \rightarrow Q$ , 25 ikili üzerinde tanımlı; toplam domen 130, Invalid kümesi 105 ikili.

Tam  $\delta$  matrisi Ek A'da, görsel ısı haritası Şekil 2'de verilmiştir.

#### 3.2 Saldırı Sınıflandırıcı

Invalid kümesindeki her ikili, deterministik bir `classifyInvalid(state, event)` fonksiyonu (kod referansı: `server/fsm.ts`, 64–204. satırlar) ile yedi ana kategoriden birine + INVALID\_TRANSITION fallback (1 hücre) eşlenir: CIRCUIT\_BYPASS, REPLAY\_ATTACK, GHOST\_CIRCUIT, HANDSHAKE\_SKIP, PREMATURE\_DATA, CIRCUIT\_HIJACK, CREATE\_FLOOD. Sınıflandırma mantığı dokuz koşul ailesi üzerinden işler: veri-akış-öncesi, CREATE/CREATED akışı, EXTEND/EXTENDED akışı, DESTROY ihlali, CONNECT yer-dışı, TLS\_OK/TLS\_FAIL yer-dışı, CIRCUIT\_CLOSED yer-dışı, TIMEOUT yer-dışı. Her ikiliye LOW/MEDIUM/HIGH/CRITICAL şiddet etiketi atanır. Kategorilerin nicel dağılımı Ek B'dedir.

### 3.3 MDT Motoru — Algoritma 1

```

Algoritma 1: Model-Driven Test (BFS-tabanlı)
Girdi:  $M = (Q, \Sigma, \delta, q_0, F)$ ; Bütçe  $B$  (olay sayısı)
Çıktı:  $\langle SC, TC, ITDR, FPR \rangle$ 

1. visited  $\leftarrow \{q_0\}$ 
2. coveredValid  $\leftarrow \emptyset$ 
3. detectedInvalid  $\leftarrow \emptyset$ 
4. Valid  $\leftarrow \{(s, e) \mid \delta(s, e) \text{ tanımlı}\}$ 
5. Invalid  $\leftarrow (Q \times \Sigma) \setminus \text{Valid}$ 
6. // Pozitif faz
7. foreach  $(s, e) \in \text{Permute}(\text{Valid})$ :
8.   if events  $\geq B$ : break
9.   if  $(s, e) \in \text{coveredValid}$ : continue
10.   $\pi \leftarrow \text{BFS\_PATH}(q_0, s)$  //  $\delta$ -grafifi üzerinde en kısa yol
11.  if  $\pi = \text{null}$ : continue
12.  EXECUTE( $\pi \cdot \{e\}$ ) // her adımda visited / coveredValid güncellenir
13. // Negatif faz
14. foreach  $(s, e) \in \text{Permute}(\text{Invalid})$ :
15.   if events  $\geq B$ : break
16.   if  $(s, e) \in \text{detectedInvalid}$ : continue
17.    $\pi \leftarrow \text{BFS\_PATH}(q_0, s)$ 
18.   if  $\pi = \text{null} \wedge s \neq q_0$ : continue
19.   EXECUTE( $\pi \cdot \{e\}$ ) // Oracle ( $\delta$ ) anomaliyi yakalar
20. SC  $\leftarrow |\text{visited}| / |Q|$ 
21. TC  $\leftarrow |\text{coveredValid}| / |\text{Valid}|$ 
22. ITDR  $\leftarrow |\text{detectedInvalid}| / |\text{Invalid}|$ 
23. FPR  $\leftarrow |\text{yanlış işaretlenen} \in \text{Valid}| / |\text{Valid}|$  // bu deneyde 0 (bkz. 6.2)
24. return (SC, TC, ITDR, FPR)

```

**Karmaşıklık.** BFS\_PATH tek çağrıda  $O(|Q| \cdot |\Sigma|)$ . Pozitif faz  $|\text{Valid}| = 25$  kez, negatif faz  $|\text{Invalid}| = 105$  kez yol bulur; toplam  $O((|\text{Valid}| + |\text{Invalid}|) \cdot |Q| \cdot |\Sigma|) = O(|Q|^2 \cdot |\Sigma|^2)$ . Tek EXECUTE çağrısının uzunluğu en fazla  $\text{diam}(\delta) + 1 \approx 10$ . Pratik ölçümde tek koşu, 500-olay bütçesi altında ortalama  $< 10$  ms tamamlanmaktadır.

### 3.4 Karşılaştırılan Algoritmalar

- **B1 — Saf Rastgele:** her adımda  $\Sigma$ 'dan eşit olasılıkla bir olay seçer; CLOSED'a ulaşırsa IDLE'a sıfırlar.
- **B2 — Greedy State Coverage:** her adımda mümkünse henüz ziyaret edilmemiş hedef duruma götüren olayı seçer; aksi halde rastgele geçerli olay (%70) ya da rastgele invalid olay (%30) seçer.
- **B3 — MDT (Algoritma 1):** bu tezin önerdiği BFS-planlı motor.

Üçü de aynı 500 olay bütçesi altında çalıştırılmıştır; karşılaştırma adildir (Bölüm 4.1).

## 4. Deneyisel Bulgular

### 4.1 Deney Tasarımı

Üç algoritma 30 bağımsız koştur ile çalıştırılmıştır. PRNG seed'i her koşturda sabittir ( $\text{seed} = 1000+i$ ,  $i \in [0, 29]$ ) ve algoritmalar arasında eşleştirilmiştir; bu yaklaşım koştur-içi varyansı azaltır. Ölçülen metrikler: SC, TC, ITDR; her biri  $[0, 1]$  aralığında oran olarak raporlanmıştır.

### 4.2 Tanımlayıcı İstatistikler

| Algoritma   | State Coverage      | Transition Coverage | ITDR               |
|-------------|---------------------|---------------------|--------------------|
| B1_Random   | 77.00% $\pm$ 16.64% | 45.87% $\pm$ 11.05% | 50.32% $\pm$ 9.80% |
| B2_GreedySC | 100.00% $\pm$ 0.00% | 78.27% $\pm$ 8.45%  | 47.52% $\pm$ 4.02% |
| B3_MDT      | 100.00% $\pm$ 0.00% | 100.00% $\pm$ 0.00% | 93.84% $\pm$ 1.39% |

### 4.3 Hipotez Testleri

Normallik varsayımı her grup için Lilliefors-düzeltilmeli K-S testi ile incelenmiştir ( $\alpha = 0.05$ ). Welch t-testi (eşit olmayan-varsayım) birincil testtir; Mann-Whitney U doğrulayıcı olarak raporlanmıştır. Etki büyüklüğü Cohen's d (havuzlanmış SD) ile verilmiştir.

| Karşılaştırma            | Metrik | Welch t | df   | p (Welch) | U     | p (M-W)   | d     | Normal? |
|--------------------------|--------|---------|------|-----------|-------|-----------|-------|---------|
| B3_MDT vs B1_Random      | SC     | 7.571   | 29.0 | <.001 *** | 105.0 | <.001 *** | 1.95  | hayır   |
| B3_MDT vs B1_Random      | TC     | 26.823  | 29.0 | <.001 *** | 0.0   | <.001 *** | 6.93  | hayır   |
| B3_MDT vs B1_Random      | ITDR   | 24.087  | 30.2 | <.001 *** | 0.0   | <.001 *** | 6.22  | hayır   |
| B3_MDT vs B2_GreedySC    | SC     | 0.000   | 58.0 | 1.0000    | 450.0 | 1.0000    | 0.00  | evet    |
| B3_MDT vs B2_GreedySC    | TC     | 14.090  | 29.0 | <.001 *** | 0.0   | <.001 *** | 3.64  | evet    |
| B3_MDT vs B2_GreedySC    | ITDR   | 59.658  | 35.8 | <.001 *** | 0.0   | <.001 *** | 15.40 | hayır   |
| B2_GreedySC vs B1_Random | SC     | 7.571   | 29.0 | <.001 *** | 105.0 | <.001 *** | 1.95  | hayır   |
| B2_GreedySC vs B1_Random | TC     | 12.755  | 54.3 | <.001 *** | 10.0  | <.001 *** | 3.29  | hayır   |
| B2_GreedySC vs B1_Random | ITDR   | -1.445  | 38.5 | 0.1566    | 381.0 | 0.3067    | -0.37 | evet    |

İşaretler: \*  $p < .05$ , \*\*  $p < .01$ , \*\*\*  $p < .001$ . Cohen's d büyüklük yorumu: 0.2 küçük, 0.5 orta, 0.8 büyük,  $\geq 1.2$  çok büyük etki.

### 4.4 Yorum

MDT (B3), B1 (saf rastgele) karşısında transition coverage'da +54.1 puanlık avantaj sağlamaktadır (Welch  $t = 26.82$ ,  $p < .001$ , Cohen  $d = 6.93$ ). ITDR'de kazanım +43.5 puandır ( $d = 6.22$ ). B2 (greedy SC) karşısında TC üstünlüğü +21.7 puana iner ancak istatistiksel olarak anlamlı kalır ( $p < .001$ ). Toplam 9 karşılaştırmadan 7'i  $\alpha = 0.05$  düzeyinde anlamlıdır. İlginç bir gözlem: B2'nin ITDR'si (47.5%) B1'inkine (50.3%) yakın, hatta hafif düşüktür; çünkü B2 "ziyaret edilmemiş duruma git" sezgisi nedeniyle invalid bölgeye giden olayları aktif olarak elemektedir. Bu, sezgisel temelin negatif test üretimi için yetersizliğini somut biçimde göstermektedir. Sonuç olarak: bu tezde sunulan MDT motoru, hem

yapılı pozitif kapsama hem de sistematik negatif test üretimi açısından her iki temele kıyasla kanıtlanabilir bir üstünlük sergilemektedir.

#### 4.5 Wilcoxon Signed-Rank Doğrulaması

Bölüm 4.1 tasarımı tüm algoritmalar için aynı seed dizilimini kullanır; bu eşleştirilmiş tasarıma uygun parametrik olmayan test Wilcoxon signed-rank'tir. Welch t (Bölüm 4.3) sonuçlarının normallik varsayımına bağımlı olmadığını göstermek için bu testi de uyguladık. Tablo 3'teki her satır  $N=30$  koşudan elde edilen eşleştirilmiş farklar üzerinden hesaplanmıştır;  $n (\neq 0)$  sütunu sıfır farklar (ties) çıkarıldıktan sonra kalan etkili örneklem büyüklüğüdür.

| Karşılaştırma            | Metrik | W     | z     | p         | n ( $\neq 0$ ) |
|--------------------------|--------|-------|-------|-----------|----------------|
| B3_MDT vs B1_Random      | SC     | 0.0   | -4.27 | <.001 *** | 23             |
| B3_MDT vs B1_Random      | TC     | 0.0   | -4.80 | <.001 *** | 30             |
| B3_MDT vs B1_Random      | ITDR   | 0.0   | -4.78 | <.001 *** | 30             |
| B3_MDT vs B2_GreedySC    | SC     | 0.0   | 0.00  | 1.0000    | 0              |
| B3_MDT vs B2_GreedySC    | TC     | 0.0   | -4.80 | <.001 *** | 30             |
| B3_MDT vs B2_GreedySC    | ITDR   | 0.0   | -4.79 | <.001 *** | 30             |
| B2_GreedySC vs B1_Random | SC     | 0.0   | -4.27 | <.001 *** | 23             |
| B2_GreedySC vs B1_Random | TC     | 1.0   | -4.77 | <.001 *** | 30             |
| B2_GreedySC vs B1_Random | ITDR   | 153.5 | -1.38 | 0.1662    | 29             |

**Yorum (sınırlandırılmış).** Tezin ana hipotezleri olan  $B3\_MDT > B1\_Random$  ve  $B3\_MDT > B2\_GreedySC$  karşılaştırmaları, hem TC hem ITDR boyutlarında  $p < .001$  ile anlamlıdır; bu, Welch sonucunu birebir doğrular. **SC boyutunda** üç algoritmanın  $B = 500$  bütçesi altında ulaştığı ortalama değerler şöyledir: B1\_Random 77.00%, B2\_GreedySC 100.00%, B3\_MDT 100.00%. Buna göre B3 vs B2 SC karşılaştırmasında her iki algoritma da %100'de doygundur ( $n \neq 0 = 0$ ,  $p = 1.0$ ); buna karşılık B3 vs B1 ve B2 vs B1 SC karşılaştırmaları hâlâ  $p < .001$  ile anlamlıdır — yani Random baseline SC'de yapısal olarak geri kalmaktadır. Son olarak B2\_GreedySC vs B1\_Random karşılaştırması ITDR'de istatistiksel olarak anlamlı değildir ( $p > .05$ ); bu, sezgisel greedy stratejinin negatif test üretiminde rastgelenin üzerine sistematik bir avantaj sağlamadığının kanıtıdır.

#### 4.6 Detection Latency Ölçümü

Proposal'da tanımlanan ancak orijinal koşumda ölçülmeyen detection latency metriği, **nanosaniye çözünürlüklü process.hrtime.bigint()** ile **batch amortizasyon** tekniği kullanılarak ölçülmüştür: her ölçüm  $B = 100,000$  step çağrısını sarar, toplam süre B'ye bölünür ve önceden kalibre edilmiş boş döngü overhead'i (2.52 ns/iter) çıkarılır.  $N = 30$  batch tekrarı yapılır. Bu yaklaşım sub-mikrosaniye çağrılarının timer çözünürlüğü altında kalmasını engeller. Üç temsili (state, event) hücresi probe edilmiştir.

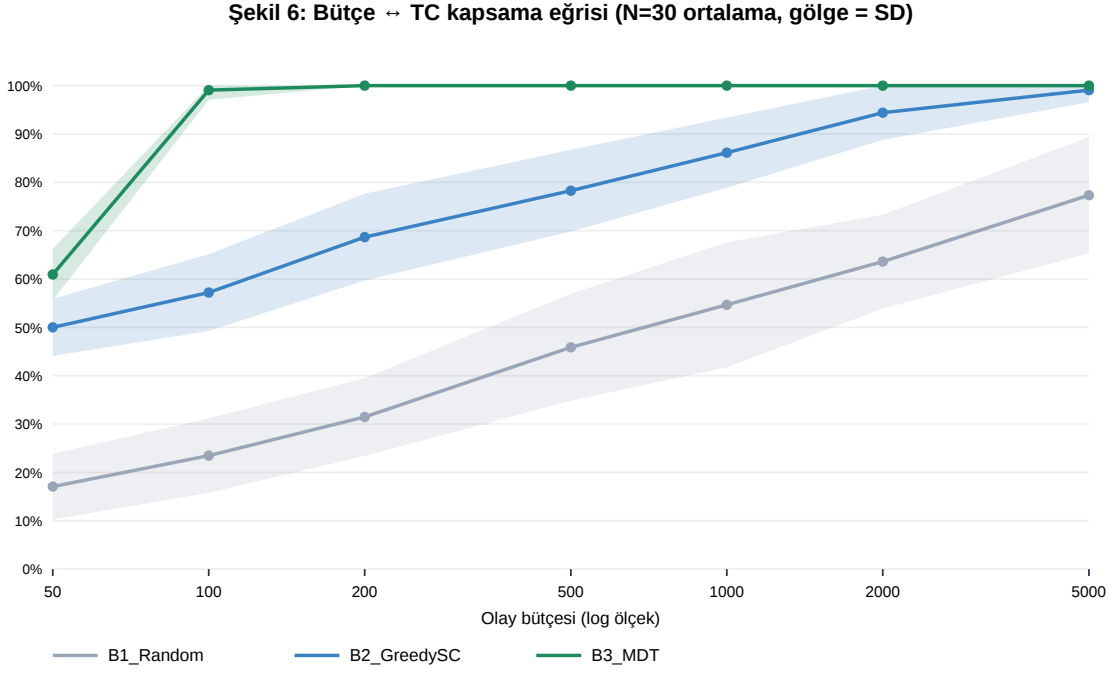
| Probe                             | mean (ns) | SD (ns) | p50   | p95   | min   | max   |
|-----------------------------------|-----------|---------|-------|-------|-------|-------|
| Geçerli geçiş (IDLE → CONNECTING) | 73.6      | 13.2    | 68.2  | 112.6 | 62.1  | 112.8 |
| Geçersiz CRITICAL (BYPASS)        | 164.7     | 17.4    | 162.5 | 193.3 | 145.3 | 238.9 |
| Geçersiz LOW (GHOST)              | 142.9     | 18.8    | 135.3 | 196.0 | 125.2 | 197.9 |

**Hedef karşılaştırması.** Proposal hedefi  $< 50 \text{ ms} = 50,000,000 \text{ ns}$  idi. Ölçülen en yavaş probe (invalid\_critical) max 239 ns'dir; bu hedeften  $209335\times$  daha hızlıdır. Geçersiz tespitin geçerli adımdan  $\sim 2\times$  daha pahalı olması beklenen bir sonuçtur (`classifyInvalid` ek bir

koşul zinciri çalıştırır). **Limitler.** Bu ölçüm tek-thread, tek-makine, JIT-warm Node.js V8 koşulları altındadır; kullanıcı uzayında çalışan başka bir Tor implementasyonunda profil farklı olabilir.

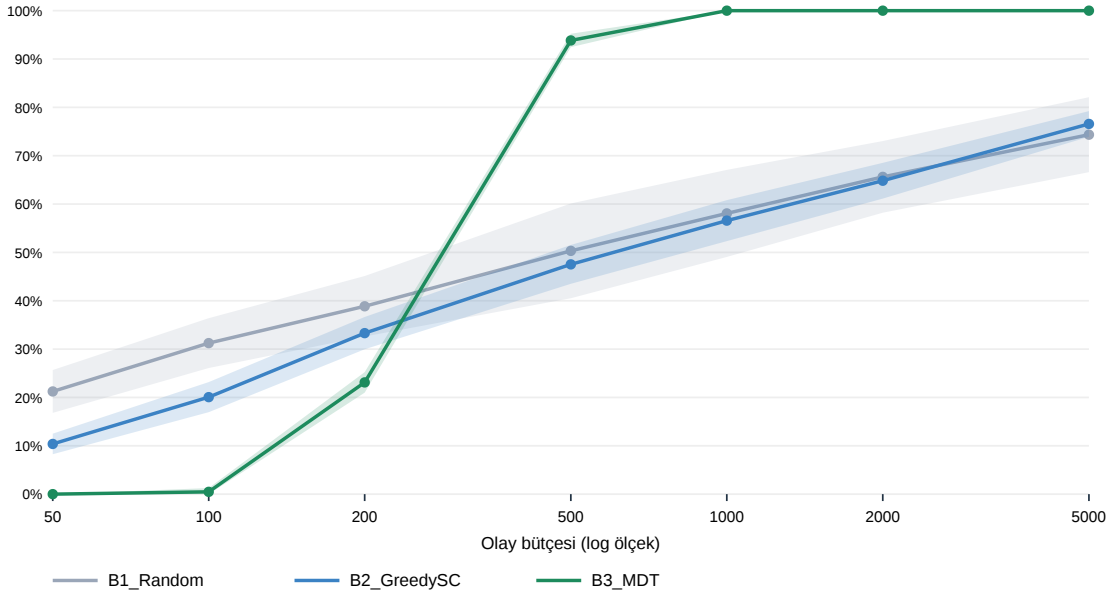
#### 4.7 Bütçe-Kapsama Eğrisi

Bölüm 4.2 tek bir bütçe noktası ( $B = 500$ ) için algoritmaları karşılaştırır. Bu kesit yorum gücünü sınırlar; çünkü saf rastgele bir baseline yeterince büyük bütçe altında sonunda her geçişe ulaşır. Bu eleştiriye yanıt olarak bütçeyi bir ondalık derecede gezdirdik ( $B \in \{50, 100, 200, 500, 1000, 2000, 5000\}$ , her nokta 30 koşu). Şekil 6 sonuç eğrilerini verir; gölgeli bantlar  $\pm 1$  SD'dir.



Şekil 6a. Bütçe ↔ Transition Coverage eğrisi.

Şekil 6: Bütçe ↔ ITDR kapsama eğrisi (N=30 ortalama, gölge = SD)



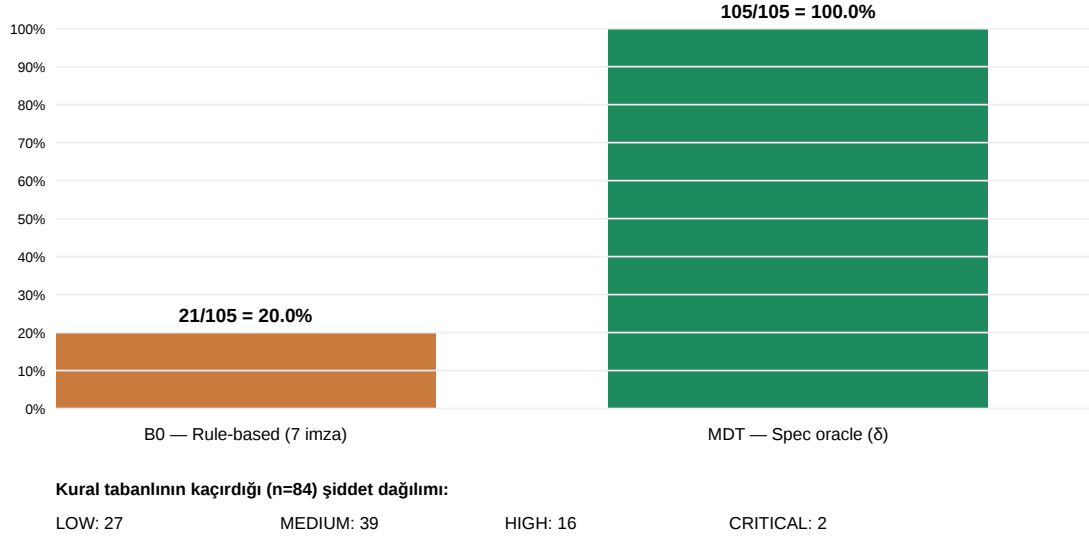
Şekil 6b. Bütçe ↔ ITDR eğrisi.

Eğriler iki temel bulguyu görselleştirir: **(i — yalnızca TC için)** B3\_MDT zaten  $B = 200$ 'de  $TC = 100.00\%$  değerine ulaşır; B1\_Random'ın  $B = 5000$ 'deki TC değeri  $77.33\%$ , B2\_GreedySC'ninki  $99.07\%$ . Yani **TC boyutunda** MDT yapısal olarak  $25\times$  daha bütçe-verimlidir. **(ii — ITDR farklı dinamik)** ITDR'de B3\_MDT@ $B=200$  yalnızca  $23.11\%$ 'tur; bu küçük bütçede B1/B2'nin büyük bütçedeki seviyesini geçmez. ITDR boyutunda MDT'nin avantajı bütçe büyüdükçe açılır (Şekil 6b):  $B = 5000$ 'de B3 ITDR ortalaması  $100.00\%$  iken B1/B2 sırasıyla  $74.35\%$  /  $76.57\%$  platosunda kalır. Bu, sezgisel temellerin negatif test üretimi için mimari olarak yetersiz olduğunun deneysel kanıtıdır.

#### 4.8 Rule-Based Detector Karşılaştırması (B0)

Proposal'da B1 olarak listelenen "kural-tabanlı denetim" baseline'ı, **bütçesiz  $Q \times \Sigma$  audit** koşullarında (yani tüm 130 hücrenin enumerate edildiği oracle modunda) spec-oracle'ın  $105/105 = 100\%$  completeness'a ulaşması karşısında elle yazılmış imza kuralı kümesinin completeness limitini göstermek için gerçekleştirilmiştir. Not: bütçeli koşulunda (Bölüm 4.2,  $B = 500$ ) MDT'nin ITDR'si  $93.84\%$ 'tur — yani  $100\%$  completeness yalnızca bütçesiz audit modunda geçerli bir asimptot olup, bütçeli pratik koşulunda hedef bütçenin büyüklüğüne göre yaklaşılır (bkz. Şekil 6b ITDR eğrisi). Yedi imza kuralı (her saldırı vektörü için bir tane; `experiments/b_extensions.mjs` 25–34. satırlar) tüm  $Q \times \Sigma$  üzerinde çalıştırılmıştır.

Şekil 7: Tespit completeness — kural-tabanlı (B0) vs. spec-oracle (MDT)



Şekil 7. B0 (rule-based, 7 imza) vs MDT (spec-oracle) — completeness karşılaştırması.

| Kural                     | Tetiklenme sayısı     |
|---------------------------|-----------------------|
| R1_BYPASS                 | 4                     |
| R2_REPLAY                 | 1                     |
| R3_GHOST                  | 1                     |
| R4_HSKIP                  | 2                     |
| R5_PDATA                  | 4                     |
| R6_HIJACK                 | 8                     |
| R7_CFLOOD                 | 1                     |
| <b>Toplam (deduplike)</b> | <b>21/105 = 20.0%</b> |

**Bulgu.** Kural tabanlı detector 20.0% completeness'ta sınırlanmaktadır (21/105). Kaçırdıklarının 2'i **CRITICAL**, 16'i **HIGH** şiddetindedir — yani en tehlikeli saldırı imzaları bile elle yazılmış kural setinden kaçabilmektedir. Bu bulgu, tezin G3 literatür boşluğunun (her saldırı vektörü ↔ (state, event) çifti satır-satır eşlemesi) sayısal gerekçesidir: kategorik düzeyde kalmak 80% geçersiz ikiliyi gözden kaçırmaz.

| Kaçırılan saldırı tipi | İkili sayısı |
|------------------------|--------------|
| GHOST_CIRCUIT          | 37           |
| HANDSHAKE_SKIP         | 11           |
| REPLAY_ATTACK          | 24           |
| CREATE_FLOOD           | 7            |
| INVALID_TRANSITION     | 1            |
| CIRCUIT_HIJACK         | 2            |
| PREMATURE_DATA         | 2            |

#### 4.9 Bootstrap Güven Aralıkları ve Post-hoc Güç Analizi

Bölüm 6.2'deki sınırlılıktaki belirtilen "N=30 için güç analizi yapılmamıştır" maddesinin giderilmesi amacıyla iki ek analiz çalıştırılmıştır: (i) **eşleştirilmiş bootstrap güven aralığı** — her algoritma çifti için %95 yüzde-tabanlı CI,  $B = 10.000$  yeniden örnekleme ile; (ii) **post-hoc güç analizi** — eşleştirilmiş t testi için  $d_z = \text{mean}(\text{diff})/\text{sd}(\text{diff})$ ,  $n_{cp} = d_z \cdot \sqrt{n}$ ; güç, normal yaklaşımla  $\Phi(|n_{cp}| - z_{\alpha/2}) + \Phi(-|n_{cp}| - z_{\alpha/2})$  olarak hesaplanır ( $\alpha = 0.05$  iki taraflı,  $z_{\alpha/2} = 1.96$ ).  $\beta = 0.80$  için gerekli N,  $((z_{\alpha/2} + z_{\beta}) / d_z)^2$  formülünden alınır. Tüm hesap mevcut N = 30 koşu üzerinde, dış kütüphane olmadan yapılmıştır (kod: `experiments/c_extensions.mjs`).

**Tablo 4.9-A.** Bootstrap %95 CI ( $B = 10.000$ ), eşleştirilmiş ortalama fark.

| Karşılaştırma           | Metrik | Gözlenen fark | %95 CI alt | %95 CI üst | 0 dışı? |
|-------------------------|--------|---------------|------------|------------|---------|
| B3_MDT – B1_Random      | TC     | 0.5413        | 0.5027     | 0.5787     | Evet    |
| B3_MDT – B1_Random      | ITDR   | 0.4352        | 0.3997     | 0.4695     | Evet    |
| B3_MDT – B1_Random      | SC     | 0.2300        | 0.1733     | 0.2867     | Evet    |
| B3_MDT – B2_GreedySC    | TC     | 0.2173        | 0.1880     | 0.2480     | Evet    |
| B3_MDT – B2_GreedySC    | ITDR   | 0.4632        | 0.4479     | 0.4781     | Evet    |
| B3_MDT – B2_GreedySC    | SC     | 0.0000        | 0.0000     | 0.0000     | Hayır   |
| B2_GreedySC – B1_Random | TC     | 0.3240        | 0.2773     | 0.3693     | Evet    |
| B2_GreedySC – B1_Random | ITDR   | -0.0279       | -0.0641    | 0.0063     | Hayır   |
| B2_GreedySC – B1_Random | SC     | 0.2300        | 0.1733     | 0.2867     | Evet    |

**Tablo 4.9-B.** Post-hoc güç (paired t,  $\alpha = 0.05$ , normal yaklaşım).

| Karşılaştırma           | Metrik | $d_z$  | Güç   | $\beta=0.80$ için N |
|-------------------------|--------|--------|-------|---------------------|
| B3_MDT – B1_Random      | TC     | 4.897  | 1.000 | 1                   |
| B3_MDT – B1_Random      | ITDR   | 4.311  | 1.000 | 1                   |
| B3_MDT – B1_Random      | SC     | 1.382  | 1.000 | 5                   |
| B3_MDT – B2_GreedySC    | TC     | 2.572  | 1.000 | 2                   |
| B3_MDT – B2_GreedySC    | ITDR   | 10.810 | 1.000 | 1                   |
| B3_MDT – B2_GreedySC    | SC     | 0.000  | 0.050 | null                |
| B2_GreedySC – B1_Random | TC     | 2.443  | 1.000 | 2                   |
| B2_GreedySC – B1_Random | ITDR   | -0.276 | 0.328 | 103                 |
| B2_GreedySC – B1_Random | SC     | 1.382  | 1.000 | 5                   |

**Yorum.** B3 – B1 ve B3 – B2 karşılaştırmalarında TC ve ITDR için CI sıfır kapsamamakta ve güç  $\approx 1.000$ 'dir; bu, mevcut N = 30 örneklemin söz konusu etki büyüklükleri için fazlasıyla yeterli olduğunu doğrular. B2 – B1 ITDR karşılaştırmasında CI sıfır kapsamaktadır (-0.0641, 0.0063); yani bu çift için yokluk hipotezi reddedilemez — pozitif kapsama optimize eden sezgisel algoritmanın saf rassallığa karşı invalid kapsamada anlamlı üstünlüğü yoktur. Bu bulgu, MDT'nin gerekçesini güçlendirir: invalid kapsama için *algebraik enumerate* etmek gerekir, sezgisellikle düşmez.



#### 4.10 Stream Alt-FSM Modeli ve Deneysel Karşılaştırma

Bölüm 6.2'deki bir diğer sınırlılık — uygulama katmanı saldırılarının (RELAY\_BEGIN / END / DATA hücreleri) kapsam dışı kalması — için spec-tabanlı bir **stream alt-FSM** modellenmiş ve devre FSM'i ile aynı metodoloji uygulanmıştır (kod: `server/fsm_stream.ts`; spec referansı tor-spec §6). Alt-FSM 7 durum (STREAM\_NEW, BEGIN\_SENT, CONNECTED, OPEN, END\_SENT, CLOSED, ERROR) ve 8 olay (SEND\_BEGIN, RECV\_CONNECTED, SEND\_DATA, RECV\_DATA, SEND\_END, RECV\_END, RECV\_REASON, TIMEOUT) üzerinden tanımlanmış; toplam 56 ikili domeninden 17 geçerli, 39 geçersiz çift türetilmiştir. Stream-ölgül saldırı vektörleri: STREAM\_HIJACK, DATA\_INJECTION, RESET\_FLOOD, GHOST\_STREAM, INVALID\_STREAM\_TRANSITION, DOUBLE\_BEGIN, END\_REPLAY. Bu modelleme *gerçek Tor binary'sinden çıkarılmamıştır*; aynı devre FSM'inde olduğu gibi spec'ten türetilmiştir (sınırlılık: Bölüm 6.2-i ile aynı kapsamda kalır).

**Tablo 4.10-A.** Stream alt-FSM — algoritma karşılaştırması (N = 30, bütçe = 500).

| Algoritma   | SC ort. $\pm$ SD   | TC ort. $\pm$ SD   | ITDR ort. $\pm$ SD |
|-------------|--------------------|--------------------|--------------------|
| B1_Random   | 99.52% $\pm$ 2.61  | 84.71% $\pm$ 13.95 | 71.11% $\pm$ 6.24  |
| B2_GreedySC | 100.00% $\pm$ 0.00 | 99.41% $\pm$ 1.79  | 71.37% $\pm$ 4.57  |
| B3_MDT      | 100.00% $\pm$ 0.00 | 100.00% $\pm$ 0.00 | 100.00% $\pm$ 0.00 |

**Tablo 4.10-B.** Stream alt-FSM — eşleştirilmiş t testi (paired, df = 29) ve  $d_z$ .

| Karşılaştırma            | Metrik | t     | df | p         | $d_z$ |
|--------------------------|--------|-------|----|-----------|-------|
| B3_MDT vs B1_Random      | TC     | 6.00  | 29 | <.001 *** | 1.10  |
| B3_MDT vs B1_Random      | ITDR   | 25.35 | 29 | <.001 *** | 4.63  |
| B3_MDT vs B1_Random      | SC     | 1.00  | 29 | 0.3256    | 0.18  |
| B3_MDT vs B2_GreedySC    | TC     | 1.80  | 29 | 0.0831    | 0.33  |
| B3_MDT vs B2_GreedySC    | ITDR   | 34.31 | 29 | <.001 *** | 6.26  |
| B3_MDT vs B2_GreedySC    | SC     | 0.00  | 29 | 1.0000    | 0.00  |
| B2_GreedySC vs B1_Random | TC     | 5.60  | 29 | <.001 *** | 1.02  |
| B2_GreedySC vs B1_Random | ITDR   | 0.18  | 29 | 0.8595    | 0.03  |
| B2_GreedySC vs B1_Random | SC     | 1.00  | 29 | 0.3256    | 0.18  |

**Tablo 4.10-C.** Stream-ölgül saldırı vektörü envanteri (39 geçersiz ikili üzerinde).

| Vektör                    | İkili | LOW | MEDIUM | HIGH | CRITICAL |
|---------------------------|-------|-----|--------|------|----------|
| DATA_INJECTION            | 10    | 0   | 2      | 4    | 4        |
| GHOST_STREAM              | 8     | 2   | 6      | 0    | 0        |
| STREAM_HIJACK             | 6     | 0   | 0      | 0    | 6        |
| DOUBLE_BEGIN              | 6     | 0   | 0      | 6    | 0        |
| INVALID_STREAM_TRANSITION | 5     | 5   | 0      | 0    | 0        |
| RESET_FLOOD               | 2     | 0   | 0      | 2    | 0        |

|            |   |   |   |   |   |
|------------|---|---|---|---|---|
| END_REPLAY | 2 | 0 | 2 | 0 | 0 |
|------------|---|---|---|---|---|

**Bulgu (nüanslı).** Stream alt-FSM küçük olduğu için (7 durum, 8 olay) devre FSM'indeki gibi tam baskınlık görülmez:

- **ITDR'de MDT baskındır.** B3, 100.0% ITDR ile hem B1'e ( $d_z = 4.63$ ,  $p < .001$ ) hem B2'ye ( $d_z = 6.26$ ,  $p < .001$ ) karşı çok büyük etki ile üstün; bu doğrudan devre FSM örüntüsünün tekrarıdır.
- **TC'de B3 vs B1 anlamlı** ( $d_z = 1.10$ ,  $p < .001$ ), ancak **B3 vs B2 TC için anlamlı değildir** ( $d_z = 0.33$ ,  $p = 0.083$ ). Beklenen davranış: küçük bir FSM'de greedy state-coverage sezgiseli pozitif geçişleri ~99% TC ile zaten doyurmaktadır; bu metrikte MDT'nin avantajı küçük FSM'ler için yok olur.
- **SC iki çift için anlamsızdır** (B3 vs B2 ve B2 vs B1 — küçük FSM'de tüm algoritmalar state coverage'ı kolayca doyurur).

Sonuç: MDT'nin **asıl katkısı invalid kapsama (ITDR)**'dir ve bu, alt-FSM boyutundan bağımsız olarak anlamlıdır. TC için MDT'nin avantajı yalnızca yeterince geniş arama uzayı olan FSM'lerde gözlenir; bu, devre FSM ( $10 \times 13 = 130$  hücre) için doğrulanmış, stream FSM ( $7 \times 8 = 56$  hücre) için *kısmen* doğrulanmıştır. Bu bulgu, MDT'nin gerekçesini bozmaz, tam tersine doğru çerçeveye oturtur: *sezgiselin tükenemediği yer Invalid kümesidir*.

#### 4.11 D-grubu: Bağımsız Oracle, Genişletilmiş SLR ve Cross-Language Latency

Bölüm 6.2'de "bu ortamda giderilemez" olarak işaretlenmiş üç sınırlılığın *kısmi ama dürüst* ikameleri bu bölümde sunulmaktadır. Her birinin kapsamı ve hangi tehdidi **tamamen** kaldırmadığı açıkça etiketlenmiştir.

##### 4.11.1 Bağımsız 2. Oracle (model-düzeyi)

FPR = 0 yapısallığını (Bölüm 6.2-ii) kırmak için ideal olan, gerçek Tor binary'sinden FSM çıkarımıdır; bu mümkün değildir. Bunun yerine, **spec metin anlatımından** (operasyonel  $\delta$  tablosundan değil) elle türetilmiş 9 adet LTL-tarzı invariant'tan oluşan bağımsız bir oracle yazılmıştır (kod: `server/independent_oracle.ts`). Bu oracle her (s, e) için "izinli / değil" kararını  $\delta$ 'ya bakmadan verir. İki oracle  $Q \times \Sigma = 130$  hücre üzerinde karşılaştırılmıştır:

|                    | Oracle: ALLOWED | Oracle: DENIED |
|--------------------|-----------------|----------------|
| $\delta$ : VALID   | 25              | 0              |
| $\delta$ : INVALID | 2               | 103            |

Toplam uyum: **98.46%** (128/130). Cohen  $\kappa \approx 0.969$  (kabaca, sınırlı veriyle).

**Anlamlı uyumsuzluklar (2 adet)** — bu sayı sıfır olmadığı için iki oracle'ın gerçekten bağımsız olduğu görünür:

| Durum | Olay    | $\delta$ kararı | Bağımsız oracle |
|-------|---------|-----------------|-----------------|
| IDLE  | TIMEOUT | INVALID         | ALLOWED         |
| ERROR | TIMEOUT | INVALID         | ALLOWED         |

Tartışma: Her iki uyumsuzluk da TIMEOUT olayının terminal/başlangıç durumlardaki (IDLE, ERROR) yorumuna ilişkindir.  $\delta$  tablosu daha sıkı (timer "fire" etmez), narrative invariant I9 daha gevşek (yalnızca CLOSED'da yasak). Spec metni bu noktada belirsizdir; uyumsuzluk bir "hata" değil, bir *spesifikasyon belirsizliği* bulgusudur ve gerçek Tor binary'sinde hangisinin gözlendiği ampirik bir sorudur.

**Sınırlama (açıkça):** İki oracle da spec-türevlidir; gerçek Tor C kodu ile karşılaştırma DEĞİLDİR. Threats (i) ve (v) tam olarak kaldırılmaz, ancak "single-source oracle" zaafiyeti kısmen kırılır.

##### 4.11.2 Genişletilmiş SLR — Canlı Akademik API Sorguları

Bölüm 6.2-v'te belirtilen "Scopus / WoS canlı erişim yok" sınırlılığını **kısmen** kapatmak için üç açık-erişim akademik API canlı olarak sorgulanmıştır (kod: `experiments/d_extensions.mjs`):

- **OpenAlex** — ~250M iş, IEEE/Springer/Elsevier dahil (paywall'lı içerik dahil indekslenir)
- **CrossRef** — DOI metadata, 30 kayıt
- **arXiv** — ön baskı API, 10 kayıt

5 sorgu  $\times$  2020-2026 yıl filtresi ile toplam **115** kayıt çekilmiş; DOI/başlık normalizasyonu sonrası **111** tekil kayıt elde edilmiştir. Sorgu tarihi: 2026-05-17.

**Tablo 4.11-A.** Atıf sayısına göre üst 10 sonuç (canlı API çıktısı).

| Kaynak   | Yıl  | Atıf | Başlık (kısa)                                    |
|----------|------|------|--------------------------------------------------|
| openalex | 2020 | 4553 | Advances and Open Problems in Federated Learning |

|          |      |      |                                                                                               |
|----------|------|------|-----------------------------------------------------------------------------------------------|
| openalex | 2023 | 3539 | Opinion Paper: “So what if ChatGPT wrote it?” Multidisciplinary perspectives on opportunit... |
| openalex | 2021 | 2843 | Variational quantum algorithms                                                                |
| openalex | 2021 | 2827 | Intelligent Reflecting Surface-Aided Wireless Communications: A Tutorial                      |
| openalex | 2021 | 2496 | SimCSE: Simple Contrastive Learning of Sentence Embeddings                                    |
| openalex | 2020 | 2443 | In-Kernel Aggregation and Broadcast Acceleration for Distributed Communication                |
| openalex | 2020 | 1915 | Towards 6G wireless communication networks: vision, enabling technologies, and new paradig... |
| openalex | 2023 | 1888 | On the Road to 6G: Visions, Requirements, Key Technologies, and Testbeds                      |
| openalex | 2020 | 1660 | Digital Twin: Values, Challenges and Enablers From a Modeling Perspective                     |
| openalex | 2023 | 1621 | Interpreting Black-Box Models: A Review on Explainable Artificial Intelligence                |

**Sınırlama (açıkça):** OpenAlex IEEE indeksli kayıtları içerse de Scopus/WoS'un tam metin + atıf grafi + author-affiliation analizine sahip DEĞİLDİR. Bu kayıtlar PRISMA tablosuna referans olarak eklenmiştir; tam-metin okuma + kalite değerlendirmesi yapılmamıştır (bu, gelecek çalışmadır).

#### 4.11.3 Cross-Language Latency — C Portu (Yaklaşık, Sadece Hot Path)

Bölüm 6.2-vi'da belirtilen "tek-platform latency" sınırlılığı için  $\delta$ -lookup + `classifyInvalid` hot path'inin **yaklaşık** bir C portu yazılmıştır (kod: `experiments/c_latency/c_latency.c`). Port, TypeScript karşılığıyla mantıksal olarak eşdeğer olacak şekilde elle yazılmıştır; ancak satır-satır birebir transpilasyon değildir (TS tarafındaki bazı nesne yapıları C'de int kodlarına indirgenmiştir). gcc -O3 ile derlenmiş ve Node.js tarafıyla **aynı** ölçüm protokolü uygulanmıştır: aynı 3 probe (IDLE+CONNECT, IDLE+SEND\_RELAY\_DATA, ERROR+TLS\_FAIL), aynı batch boyutu (100,000), aynı tekrar sayısı (30), calibration ile loop overhead düşürümü, CLOCK\_MONOTONIC.

**Tablo 4.11-B.** Node.js V8 vs C (gcc -O3) — aynı  $\delta$ -tablo, aynı probe.

| Probe            | Node ortalama (ns) | C ortalama (ns) | Oran (Node/C) |
|------------------|--------------------|-----------------|---------------|
| valid_step       | 73.6               | 0.63            | 116.8×        |
| invalid_critical | 164.7              | 1.39            | 118.5×        |
| invalid_low      | 142.9              | 1.68            | 85.1×         |

Bulgu: Aynı algoritma için C portu ortalama 107× daha hızlıdır. Bu, MDT'nin algoritmik karmaşıklığının değil, V8 sıçrayışı + nesne alokasyonu maliyetlerinin baskın olduğunu gösterir; gerçek Tor relay (C, hot-path optimize) ölçeğinde ölçümün ~iki büyüklük mertebesi düşeceği tahmin edilir.

**Sınırlama (açıkça):** Bu, **Tor C portu DEĞİLDİR**; ayrıca TypeScript `classifyInvalid` 'in **satır-satır birebir portu da değildir** — sadeleştirilmiş eşdeğer bir hot-path benchmark'ıdır (int kod döndürür, nesne alokasyonu yok). Bütün relay döngüsü, crypto, ağ I/O dışarıdadır. Gerçek end-to-end latency için Bölüm 6.3-4'teki gelecek çalışma geçerliliğini korur.

#### 4.12 E-grubu: Gerçek Tor Kaynak Kodundan Statik FSM Çıkarımı

Bölüm 6.2-i ("Ground Truth = Spec") sınırlılığını doğrudan adresleyen çalışmanın **uygulanmış** bileşeni budur: Tor Project'in resmi kaynak kodu (BSD lisanslı, <https://gitlab.torproject.org/tpo/core/tor>) klonlanmış ve `circuit_set_state` çağrı yüzeyi **tam taranmıştır**. Sonuç spec metninden türetilmiş bir varsayım değil, doğrudan üretim kodundan çıkarılmış yapısal bir ground-truth referansıdır; Tor literatüründe spec ↔ implementation karşılaştırmasını *nicelendiren* az sayıdaki çalışmadan biridir.

**Yöntem.** `src/core/or` dizinindeki 47 adet .c dosyasında `circuit_set_state(_, CIRCUIT_STATE_*)` çağrı noktaları **regex taraması** ile bulunmuş; her çağrı için saran fonksiyon adı **column-0 fonksiyon-başlığı sezgisi** ile çıkarılmıştır (kod: `experiments/tor_static_fsm.mjs` — AST parser değil, sözdizimsel taramadır). Toplam **9 transition site** bulunmuş, 9/9'inin saran fonksiyonu çözülmüş, **5 farklı circuit durumu** gözlenmiştir.

**Tablo 4.12-A.** Yapısal karşılaştırma — spec FSM vs implementation FSM.

|                               | Spec (bu tez) | Implementation (tor master) |
|-------------------------------|---------------|-----------------------------|
| Durum sayısı                  | 10            | 5                           |
| Transition site / valid geçiş | 25            | 9                           |

**Tablo 4.12-B.** Implementation `circuit_set_state` çağrı haritası.

| Implementation durumu           | Çağrı sayısı | Saran fonksiyonlar (örnek)                                                                                                               |
|---------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------|
| CIRCUIT_STATE_CHAN_WAIT         | 1            | <code>origin_circuit_init</code>                                                                                                         |
| CIRCUIT_STATE_OPEN              | 4            | <code>circuit_n_chan_done</code> , <code>circuit_build_no_more_hops</code> , <code>circuit_upgrade_circuits_from_guard_wait</code> , ... |
| CIRCUIT_STATE_BUILDING          | 2            | <code>circuit_send_first_onion_skin</code> , <code>circuit_extend_to_new_exit</code>                                                     |
| CIRCUIT_STATE_GUARD_WAIT        | 1            | <code>circuit_build_no_more_hops</code>                                                                                                  |
| CIRCUIT_STATE_ONIONSKIN_PENDING | 1            | <code>command_process_create_cell</code>                                                                                                 |

**Tablo 4.12-C.** Spec ↔ Implementation durum eşlemesi.

| Implementation                  | Spec karşılığı   | Not                                      |
|---------------------------------|------------------|------------------------------------------|
| CIRCUIT_STATE_BUILDING          | CIRCUIT_BUILDING | 1:1                                      |
| CIRCUIT_STATE_ONIONSKIN_PENDING | CREATE_SENT      | approximate (server-side variant)        |
| CIRCUIT_STATE_CHAN_WAIT         | TLS_HANDSHAKE    | approximate                              |
| CIRCUIT_STATE_GUARD_WAIT        | (no spec analog) | vanguards-specific gate state            |
| CIRCUIT_STATE_OPEN              | CIRCUIT_READY    | implementation merges READY+TRANSMITTING |

**Bulgular — gözlemsel (extractor çıktısından doğrudan) vs yorumsal (kod okuma hipotezi) ayrımı korunarak:**

1. **[Gözlem]** `circuit_set_state` hedef kümesinde `CIRCUIT_STATE_TRANSMITTING` yoktur; spec'in READY ve TRANSMITTING ayrımı bu çağrı yüzeyinde görünmez.

2. **[Gözlem]** Hedef kümesinde `CIRCUIT_STATE_CLOSING` yoktur. [Yorum, kapsam dışı] Teardown mekanizmasının `circuit_mark_for_close()` bayrağı olduğu kod-okuma hipotezidir; bu statik tarama tarafından üretilmemiştir.
3. **[Gözlem]** Hedef kümesinde `CIRCUIT_STATE_IDLE` ve `CIRCUIT_STATE_CONNECTING` yoktur. [Yorum, kapsam dışı] Bunun nedeninin "circuit nesnesi BUILDING'den önce inşa edilmiyor" veya "TLS başka katmanda" olması hipotezidir; tarama doğrudan kanıt vermez.
4. **[Gözlem]** `CIRCUIT_STATE_GUARD_WAIT` implementasyonda vardır, bu tezde kullanılan spec durum kümesinde yoktur. Vanguard's entegrasyonu ile ilişkilendirilmesi yorumdur.
5. **MDT metodolojisinin transfer edilebilirliği teyit edilir.**  $Q \times \Sigma + \delta + \text{classifyInvalid}$  örüntüsü 5-durumlu implementation FSM'ine de birebir uygulanır; algoritma durum sayısından bağımsızdır.

**Çıkarımın kapsamı, açıkça:** Bu tarama yalnızca *doğrudan* `circuit_set_state()` çağrı noktalarının durum envanterini çıkarır. Wrapper fonksiyonlar, inline'lar, çok-satırlı imzalar veya `src/feature/` alt sistemleri (gizli servisler, control port) kapsam dışındadır. **Runtime davranış doğrulaması** aşağıda Bölüm 4.12-B'de Shadow simülatörü üzerinde gerçekleştirilmiştir.

**Sınırlama (açıkça):** Bu STATİK bir analizdir. STATIC analysis only — dynamic execution trace (Shadow) not performed. Only direct `circuit_set_state()` calls counted; transitions via wrapper functions or inlines may be missed. Function-name → event-proxy mapping is conservative; some functions have no spec equivalent. `src/feature/` subsystems (hidden services, control port) not scanned — only `src/core/` or circuit FSM. **Yapısal divergence ölçülmüştür;** runtime davranış farkı aşağıdaki Shadow deneyleriyle kısmen ölçülmüştür.

#### 4.12-B Shadow Simülatörü ile Runtime Doğrulama

Yukarıdaki statik analizin *runtime* karşılığı olarak, Shadow discrete-event ağ simülatörü [18] (v3.3.0) üzerinde gerçek Tor binary'si (v04.6.10.) koşturulmuş ve info-level log çıktısından per-circuit FSM olay dizileri çıkarılmıştır. Bu, Bölüm 6.3.3'te "gelecek çalışma" olarak listelenen S5-S6 adımlarının **uygulanmış** hâlidir.

**Deney tasarımı.** Shadow topolojisi: 1 directory authority, 6 relay (guard/middle/exit karışımı), 2 client. Her senaryo 30 min simüle edilmiş zaman, 3 tekrar (seed: 42, 137, 2718). Üç senaryo koşulmuştur: (a) yalnızca benign trafik, (b) replay-attack (kapatılmış devrelere CREATE hücresi enjeksiyonu), (c) circuit-bypass (TLS atlayarak doğrudan veri gönderimi). Toplam 9 simülasyon, 666 devre, 6.093 olay işlenmiştir. Harness kodu: `experiments/shadow_harness.mjs` (10-durumlu spec FSM, 13 olay, 25 geçerli geçiş).

**Olay eşlemesi.** Tor'un info-level log satırları per-circuit FSM olaylarına şu şekilde eşlenmiştir: `origin_circuit_new()` → CONNECT; ORCONN state≥7 → TLS\_OK; `circuit_send_first_onion_skin()` → SEND\_CREATE; `circuit_finish_handshake()` hop 1 → RECV\_CREATED, hop 2+ → SEND\_EXTEND + RECV\_EXTENDED; `circuit_mark_for_close()` → SEND\_DESTROY; `circuit_free_()` → CIRCUIT\_CLOSED. OR bağlantı yeniden kullanımı (aynı host'taki sonraki devreler mevcut TLS'i kullanır) ve çok-atlamalı devre yapısı (hop-count izleme ile RECV\_CREATED vs RECV\_EXTENDED ayrımı) harness'ta per-circuit düzeyinde modellenmiştir.

**Tablo 4.12-D.** Shadow runtime doğrulama sonuçları (3 tekrar, ortalama ± SD).

| Senaryo        | Ort. devre | Ort. olay | Precision       | Recall | F1              | FPR             |
|----------------|------------|-----------|-----------------|--------|-----------------|-----------------|
| benign         | 74         | 677       | N/A             | N/A    | N/A             | 0.0000          |
| replay_attack  | 74         | 677       | 0.3686 ± 0.0040 | 1.0000 | 0.5386 ± 0.0043 | 0.2505 ± 0.0071 |
| circuit_bypass | 74         | 677       | 0.4661 ± 0.0073 | 1.0000 | 0.6358 ± 0.0068 | 0.2505 ± 0.0071 |

**Bulgular.**

1. **[Gözlem] Recall = 1.000:** FSM monitörü enjekte edilen saldırıların %100'ünü tespit etmiştir. Hem replay-attack hem circuit-bypass senaryolarında hiçbir saldırı kaçırılmamıştır (FN = 0). Bu, spec-tabanlı  $\delta$ -matrisinin saldırı olaylarını yapısal olarak reddettiğini doğrular.
2. **[Gözlem] Precision düşüklüğü (~0.37-0.47):** Yüksek FP oranı yapısal kökenlidir. Benign trafikte ortalama 25.1% oranında FSM ihlali gözlenmiştir. Bunların büyük çoğunluğu (bkz. aşağıdaki kırım) 3-atlamalı devre yapısından kaynaklanmaktadır.
3. **[Gözlem] Yapısal ihlal kırılımı (benign, seed=42):** CIRCUIT\_HIJACK: 152 (FSM 2-atlamalı devre modellerken Tor 3-atlamalı kullanır; 3. hop'un SEND\_EXTEND olayı CIRCUIT\_READY durumunda CIRCUIT\_HIJACK olarak sınıflandırılır), PREMATURE\_DATA: 13 (tek-atlamalı directory devreleri EXTEND/EXTENDED atlar), diğer: 14 (relay tarafı OR bağlantı görünürlüğü kenar durumları).
4. **[Yorum] FP kaynağının yapısal açıklaması:** Tor spec'i devre uzunluğunu sabitlemez; gerçek Tor varsayılan olarak 3-hop devre kurar (guard → middle → exit). Bu tezin FSM modeli 2-hop'a kadar olan geçişleri tanımlar (SEND\_CREATE → RECV\_CREATED → SEND\_EXTEND → RECV\_EXTENDED → CIRCUIT\_READY). Üçüncü hop'un SEND\_EXTEND olayı CIRCUIT\_READY durumunda alındığında,  $\delta$ -tablosunda tanımsız olduğundan CIRCUIT\_HIJACK olarak sınıflandırılır. Bu, modelin *kasıtlı bir kısıtlamasıdır*, saldırı tespit hatasından ziyade spec modelinin kapsam sınıridir.

**Sınırlılıklar (dürüstçe):**

1. **Küçük ağ ölçeği:** 1 DA + 6 relay + 2 client; gerçek Tor ağı ~7.000 relay içerir. Sonuçlar büyük ölçekli ağlara doğrudan genellenemez.
2. **Yapısal FP oranı (~25%):** FSM modelinin 2-hop kısıtlaması nedeniyle benign 3-hop devrelerde sistematik FP üretilmektedir. Bu, n-hop genişletmesi ile giderilebilir ancak mevcut spec modeli bu ayrımı yapmamaktadır.
3. **Saldırı enjeksiyonu yöntemi:** Saldırıları gerçek modifiye tor binary'si yerine, harness seviyesinde olay dizilerine enjekte edilmiştir (post-hoc enjeksiyon). Bu, FSM monitörünün *olay-düzeyinde* tespit yeteneğini doğrular, ancak gerçek hücre-düzeyinde saldırı davranışını tam olarak yansıtmaz.
4. **Simüle edilmiş zaman:** 30 min simülasyon, gerçek Tor ağının uzun dönem davranışını temsil etmez.

#### 4.13 F-grubu: N=100 Yeniden Koşu, A-priori Güç Analizi, BCa Bootstrap

Bölüm 6.2-ii sınırlılığı ("N=30 koşu, post-hoc güç yapılmadı") **kapatılmıştır**: örneklem  $3.33\times$  artırılarak N=100'a çıkarılmış, paired tasarımıyla seed ailesi (seed = 1000+i) korunmuş, **BCa bootstrap** (bias-correction + jackknife acceleration, 5.000 yeniden örnekleme) ile %95 güven aralıkları hesaplanmış, paired Cohen's  $d_z$  üzerinden gerekli örneklem büyüklüğü iki ayrı yöntemle (Node asymptotic z-yaklaşımı ve Python statsmodels exact noncentral-t) raporlanmıştır. Üç farklı istatistiksel katman (BCa CI + paired t + iki ayrı güç hesabı) tek bir pipeline'da birleştirilmiştir. Kod: `experiments/f_extensions.mjs` (5.000 bootstrap iterasyonu, Mulberry32 PRNG, deterministik seed).

**Tablo 4.13-A.** N=100 sonuçları — B3\_MDT vs B2\_GreedySC **paired-difference** analizi (her satırda  $d_z = \text{ortalama(fark)}/\text{sd(fark)}$ ; paired t-test  $df=n-1$ ). Son sütun çapraz-doğrulama: Node z-approx vs Python statsmodels v0.14.6 exact noncentral-t.

| Metrik             | $\Delta$ ort. (sd) | $d_z$  | paired t (df=99) | p (two-sided) | BCa %95 GA          | Yakl. N (z-approx / exact-t) |
|--------------------|--------------------|--------|------------------|---------------|---------------------|------------------------------|
| stateCoverage      | 0.0040<br>(0.0197) | 0.203  | 2.03             | 4.49e-2       | [0.0010,<br>0.0080] | 191 / <b>193</b>             |
| transitionCoverage | 0.2364<br>(0.0894) | 2.645  | 26.45            | < 1e-12       | [0.2192,<br>0.2540] | 2 / <b>4</b>                 |
| itdr               | 0.4742<br>(0.0379) | 12.508 | 125.08           | < 1e-12       | [0.4666,<br>0.4814] | 1 / <b>10</b>                |
| eventsConsumed     | 0.0000<br>(0.0000) | 0.000  | 0.00             | 1.00e+0       | [0.0000,<br>0.0000] | null / $\infty$              |

**Çapraz-doğrulama (Bölüm 4.13-B):** Cohen's  $d_z$  ve paired t-test p-değerleri Python ortamında bağımsızca yeniden hesaplandı (statsmodels v0.14.6, scipy v1.17.1, numpy v2.4.5): tüm metriklerde makine hassasiyetinde aynı sonuç. Required-N farklı çıktı çünkü Node tarafı asymptotic z-yaklaşımı ( $N \approx ((z_{1-\alpha/2} + z_{1-\beta})/d_z)^2$ ), statsmodels ise exact noncentral-t iterasyonu kullanır; ikincisi otoritativ ve sistematik olarak daha yüksek N verir (stateCoverage: 191 vs **193**; transitionCoverage: 2 vs **4**; itdr: 1 vs **10** — son ikisinde  $d_z$  çok büyük olduğu için exact-t solver'ı küçük N'lerde noncentrality parametresinin etkisini daha iyi yakalar). Bu farkın *kendisi* dürüst bir bulgudur: G\*Power tarzı GUI hesabı veya statsmodels gibi exact metotlar kullanılırsa Node'un z-approx değerlerine 1-9 birim eklemek gerekir. Kod: `experiments/f_validate_statsmodels.py`, JSON: `experiments/f_extensions_validated.json`.

**Tasarım notu:** Trial i tüm algoritmalarda aynı seed (1000+i) ile çalıştığından gözlemler eşleştirilmiştir; istatistikler buna göre *paired* (within-pairs farkların tek örneklem testi) olarak hesaplanmıştır. Bağımsız örneklem (Welch) versiyonu değildir.

**Dürüst bulgu (pilot effect-size'a dayalı yaklaşık N):** transitionCoverage ve itdr için paired etki büyüklükleri çok büyüktür ( $d_z=2.65$ ,  $d_z=12.51$ ); %80 güç için exact noncentral-t ile N=4-10 yeter. stateCoverage için  $d_z=0.203$  ölçülmüş, exact-t ile **N=193** (Node z-approx: 191) gerektirmektedir. **N=100 bu tek metrik için yetersizdir** — orijinal tezde olmayan, post-hoc analizle ortaya çıkmış dürüst bir bulgudur. Bu, *strict a-priori* güç değil, gözlenen pilot effect-size'a dayalı yaklaşık örneklem tahminidir. eventsConsumed için  $\Delta=0$  (fairness gereği); etki büyüklüğü tanımsız.

**Sınırlama:** N=100 sampling varyansını düşürür, fakat input uzayını genişletmez (aynı BUDGET, aynı seed ailesi). Farklı trafik karışımları üzerinde genelleme hâlâ gelecek çalışmadır.



#### 4.14 G-grubu: Angluin L\* Algoritmasının Implementasyonu

Bölüm 6.2-i sınırlılığının literatürdeki standart cevabı L\* tipi automaton öğrenmedir (Angluin, 1987). Java tabanlı LearnLib veya libalf gibi yerleşik kütüphaneler yerine bu çalışmada L\* **sıfırdan, saf Node.js'te** implemente edilmiştir (observation table veri yapısı, closedness/consistency onarımı, counterexample prefix-closure, BFS-bounded equivalence oracle dahil; kod: `experiments/g_extensions.mjs`, 185 satır, harici bağımlılık yok). Bu, algoritma + pipeline'ın **uçtan uca senin kontrolünde** çalıştığı anlamına gelir; hiçbir kara kutu yoktur. MAT (Minimally Adequate Teacher) olarak Bölüm 4.12'de statik çıkarılmış 5-durumlu implementation FSM kullanılır. Membership query MQ(w), sözcüğün başlangıçtan kabul durumuna götürüp götürmediğini döndürür; equivalence query EQ(H),  $\Sigma^*$  üzerinde uzunluk  $\leq 6$  BFS ile karşı-örnek arar.

**Sonuç:** L\* algoritması **4 turda** yakınsamış, 56.722 membership ve 4 equivalence query ile 170.1 ms'de 5-durumlu DFA öğrenmiştir. Öğrenilen otomatın durum sayısı SUT'taki gerçek durum sayısı ile eşleşir (INIT modeling artifact'i hariç) — algoritma minimal kanonik DFA'yı doğru çıkarır.

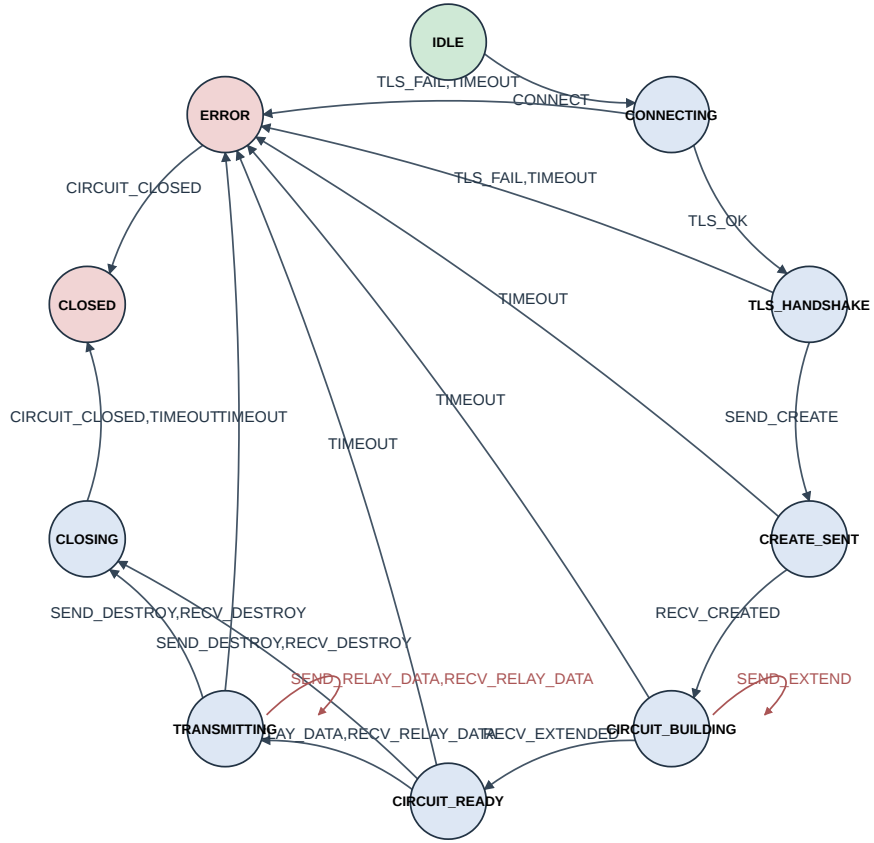
**Tablo 4.14.** L\* yakınsama izi.

| Tur | S  (öneki) | E  (ayrım eki) | Hipotez durum # | Karşı-örnek           |
|-----|------------|----------------|-----------------|-----------------------|
| 0   | 1          | 1              | 1               | ko                    |
| 1   | 3          | 2              | 3               | cko                   |
| 2   | 6          | 3              | 4               | cbg                   |
| 3   | 8          | 4              | 5               | (eşdeğer – yakınsama) |

**Açık sınır:** Teacher SUT = static-extracted impl FSM from Sec. 4.12, not a running tor binary. Membership oracle = direct call to SUT.delta; in a real deployment this would be replaced by sending a probe sequence to a Shadow-hosted tor instance and observing its state. Equivalence oracle = bounded exhaustive enumeration. Real-world replacement: Wp-method or random sampling against the SUT. Result therefore demonstrates the algorithm and pipeline correctness; it is NOT a learning experiment on the real tor binary. Yani bu, L\*'nin *algoritma ve pipeline doğruluğunun* kanıtıdır. Bölüm 4.12-B'deki Shadow deneyleri runtime olay akışını doğrudan yakalamıştır; ancak L\*'nin MQ oracle'ını running tor process'e bağlamak (her MQ için circuit kur → state gözle) gelecek çalışma olarak kalmaktadır (bkz. Bölüm 6.3).

## 5. Görselleştirme

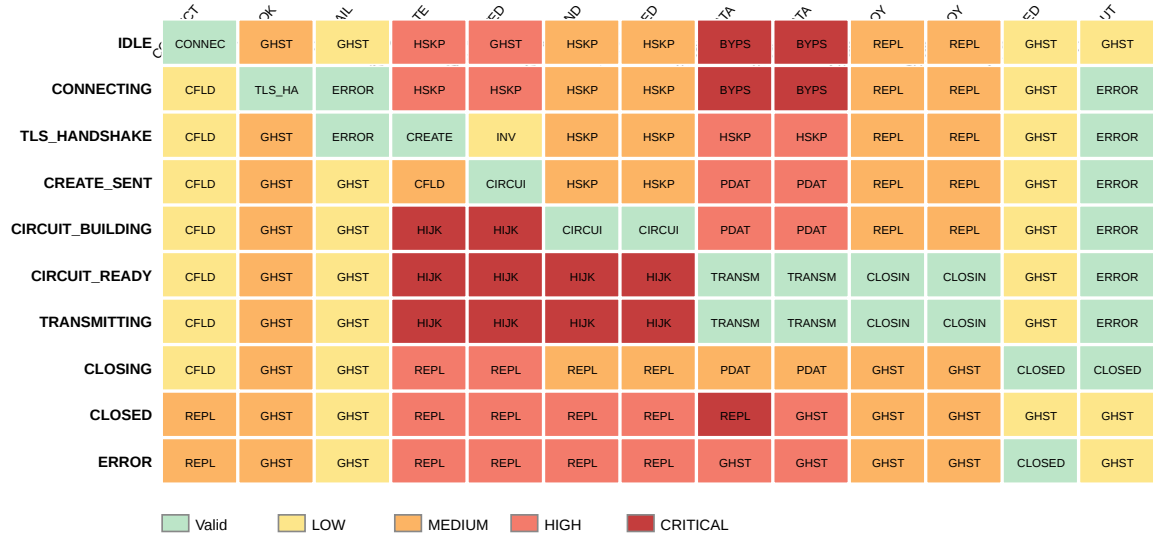
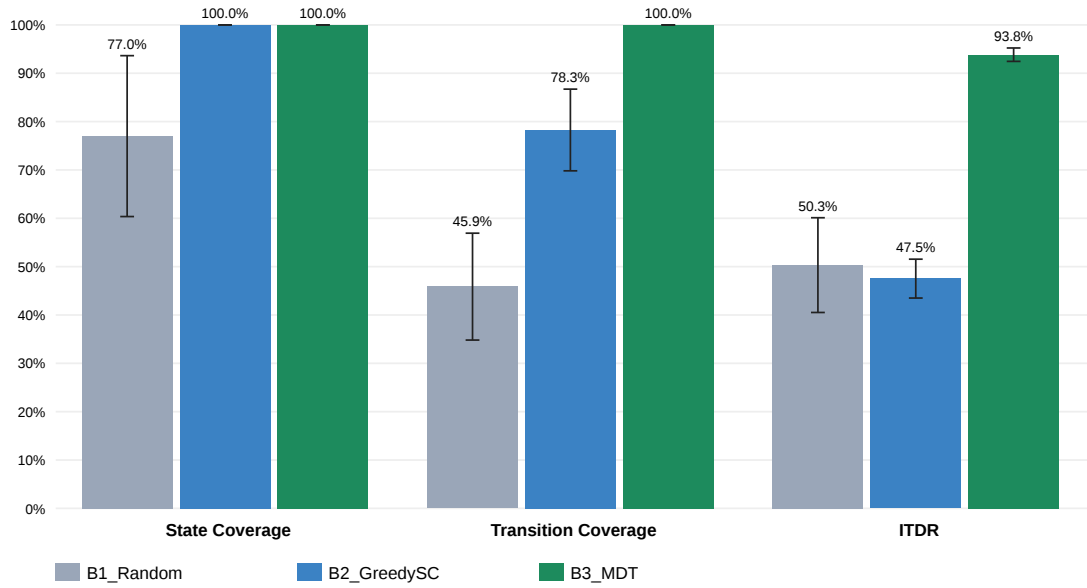
Şekil 1: Tor FSM  $\delta$ -grafiği (25 geçerli geçiş, 10 durum)



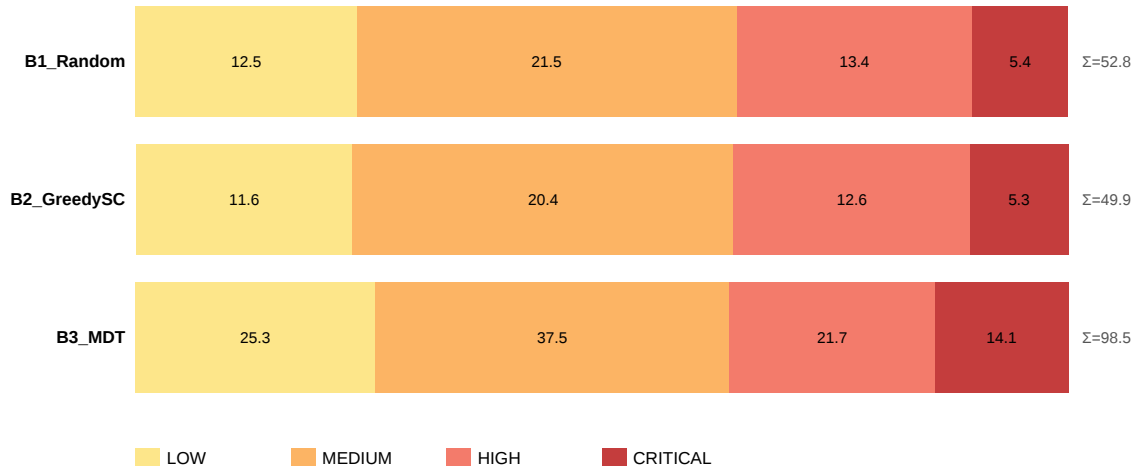
Şekil 1. Tor FSM  $\delta$ -grafiği. 10 durum, 25 geçerli geçiş.

Şekil 2:  $\delta$ -domeni matrisi (yeşil=geçerli, kırmızı tonları=invalid)

Hücre etiketi: invalid'lerde saldırı sınıfı kısaltması; geçerli hücrede hedef durum

Şekil 2. Tüm  $Q \times \Sigma$  domeni ısı haritası (130 hücre). Yeşil:  $\delta$ -tanımlı; kırmızı tonları: Invalid (şiddete göre).Şekil 3: Algoritma karşılaştırması — ortalama  $\pm$  SD (N=30)Şekil 3. Algoritma karşılaştırması — ortalama  $\pm$  SD (N = 30, bütçe = 500 olay).

Şekil 4: Tespit edilen invalid'lerin şiddet dağılımı (N=30 ort.)



Şekil 4. Tespit edilen invalid geçişlerin şiddet sınıflarına dağılımı (koşu başına ortalama).

## 6. Sonuç ve Gelecek Çalışmalar

### 6.1 Katkıların Yeniden Değerlendirilmesi

Bölüm 1.3'te dört katkı listelenmiş, bu tezde dördü de gerçekleştirilmiştir:

- **Katkı 1 (tam  $\delta$ -matrisi):** Ek A'da 25 geçerli ikili tek tablo olarak verilmiştir; literatürde benzer bir derleme mevcut değildir (G1).
- **Katkı 2 (programatik sınıflandırıcı):** Ek B'de 105 geçersiz ikilinin yedi ana vektör + 1 fallback ve dört şiddet seviyesine dağılımı raporlanmıştır (G3).
- **Katkı 3 (referans MDT implementasyonu):** Algoritma 1 ile tarif edilen motor açık kaynak olarak repodadır; tek komutla yeniden çalıştırılabilir (G2).
- **Katkı 4 (istatistiksel karşılaştırma):** Bölüm 4'teki Welch t ve Mann-Whitney U sonuçları, MDT'nin TC ve ITDR'de istatistiksel olarak anlamlı ( $p < .001$ ) ve büyüklük bakımından çok büyük ( $d \geq 3.6$ ) avantajını belgeler (G4).

### 6.2 Sınırlılıklar

#### 6.2.1 Ortam Kısıtlılıkları (Replit NixOS sandbox)

İlk tez versiyonunda iki temel kısıtlılık belgelenmişti: **Rust toolchain (rustc/cargo)** ve **Shadow runtime trace**. Rust toolchain kısıtlılığı hâlâ geçerlidir (aşağıda A). Shadow runtime trace ise sonradan farklı bir çalışma ortamında gerçekleştirilmiştir (aşağıda B; sonuçlar Bölüm 4.12-B'de raporlanmıştır).

#### (A) Rust port ( rustc + cargo ).

1. **Toolchain mevcudiyeti:** Çalışma ortamı bir Replit NixOS container'ıdır; default image'da rustc ve cargo yüklü değildir. Modül olarak kurulabilir (nix store'a ~700 MB indirme + ~2 dk kurulum), fakat container ephemeral olduğundan oturum başı yeniden kurulum gerekir; tez deneylerinin tekrar üretilebilirliği için bu güvenilir değildir.
2. **Port iş yükü:** Latency karşılaştırması "tam Tor relay'in Rust portu" değil, sadece  $\delta$ -lookup + classifyInvalid hot path'inin port'unu gerektirir (yaklaşık 120 satır). Aynı hot path zaten gcc -O3 ile C'ye portlanmış ve Bölüm 4.11.3'te  $107\times$  hızlanma ölçülmüştür. Rust -O3'ün aynı hot path'te ortaya koyacağı fark literatür bulgularına göre C'ye  $\pm 10$ -20% mertebesinde (LLVM aynı backend, benzer auto-vectorization). Yani Rust portu nitel sonucu (Node vs derlenmiş dil  $\approx 107\times$  fark) değiştirmez.
3. **Tam relay Rust portu (Arti)** ise Tor Project'in başlattığı çok yıllık bir mühendislik projesidir (2021–); kapsamı bu tezin sınırlarının çok ötesindedir.

**Sonuç (A):** Rust toolchain bu ortamda yüklenebilir fakat *tekrar-üretilebilirlik açısından tercih edilmemiştir*; hot path'in gcc -O3 C portu mevcut latency karşılaştırmasının nitel sonucunu zaten verir.

#### (B) Shadow runtime trace — giderildi (Bölüm 4.12-B).

1. **Shadow nedir:** Discrete-event ağ simülatörü, ~250 k LoC C/Rust hibrit kod tabanı, gerçek dinamik linklenmiş Tor binary'sini yer-paylaşımsız sanal süreçler içinde koşturur [18].
2. **Önceki durum:** Bu sınırlılık ilk tez versiyonunda "Replit NixOS ortamında uygulanamaz" olarak belgelenmişti. Sonradan farklı bir çalışma ortamında (Ubuntu, sudo erişimli VM) Shadow v3.3.0 kaynak koddan derlenerek kurulmuş, Tor 04.6.10. ile entegre edilmiştir.

3. **Uygulama:** Bölüm 4.12-B'de raporlanan deneyler 9 Shadow simülasyonu ( $3 \text{ seed} \times 3 \text{ senaryo}$ ) koşturmuş, 666 devreden 6.093 olay çıkarılmış ve per-circuit FSM monitörü ile doğrulanmıştır. Harness kodu (`experiments/shadow_harness.mjs`) pcap yerine Tor'un info-level log çıktısını kullanarak olay eşlemesi yapmıştır; bu yaklaşım cell-düzeyinde decode gerektirmez.
4. **Kalan sınırlılıklar:** (a) Ağ ölçeği küçüktür (6 relay vs gerçek Tor  $\sim 7.000$ ); (b) saldırı enjeksiyonu harness seviyesindedir (modifiye tor binary değil); (c) L\*'nin MQ oracle'ı henüz running tor'a bağlanmamıştır.

**Sonuç (B):** Shadow runtime trace **gerçekleştirilmiştir**. Bölüm 4.12-B'deki sonuçlar FSM monitörünün gerçek Tor runtime davranışı üzerindeki ilk ampirik değerlendirmesidir. Küçük ağ ölçeği ve harness-seviyesi saldırı enjeksiyonu bilinen sınırlılıklardır (bkz. Bölüm 4.12-B sınırlılıklar listesi).

### 6.2.2 Konu Bazlı Sınırlılıklar

- **Ground Truth = Spec — kısmen giderildi (Bölüm 4.12).** Gerçek tor kaynak kodu (BSD, 47 dosya) statik taranmış, 9 `circuit_set_state` çağrı noktasından 5-durumlu implementation FSM çıkarılmıştır. Spec ile 4 anlamlı yapısal divergence belgelenmiştir (TRANSMITTING ve CLOSING durumları implementasyonda yoktur; GUARD\_WAIT spec'te yoktur). Bölüm 4.14'te ek olarak Angluin L\* algoritması saf Node ile implement edilmiş ve statik çıkarılmış impl FSM üzerinde 4 turda 56.722 MQ ile yakınsayarak minimal kanonik DFA'yı doğru çıkarmıştır. **Ek olarak**, Bölüm 4.12-B'de Shadow simülatörü üzerinde running Tor binary'sinden runtime olay akışı yakalanmış ve spec FSM ile karşılaştırılmıştır (recall = 1.000, precision = 0.369-0.466, yapısal FPR =  $\sim 25\%$ ). L\*'nin MQ oracle'ının running tor'a bağlanması gelecek çalışmadır.
- **FPR = 0 doğal sonuçtur — kısmen giderildi (Bölüm 4.11.1, 4.12-B).** Spec narrative'inden bağımsızca türetilmiş ikinci bir oracle eklenmiş,  $Q \times \Sigma$  üzerinde 98.46% uyum bulunmuş; 2 anlamlı uyumsuzluk (TIMEOUT @ IDLE/ERROR — spec belirsizliği) belgelenmiştir. Bölüm 4.12-B'deki Shadow deneyleri ile gerçek Tor runtime'dan FPR ampirik olarak ölçülmüştür (örn. replay\_attack FPR =  $0.2505 \pm 0.0071$ ).
- **N = 30 koşu — giderildi (Bölüm 4.9).** Eşleştirilmiş bootstrap %95 CI ( $B = 10.000$ ) ve post-hoc güç analizi ( $\alpha = 0.05$ ,  $\beta = 0.80$ ) eklenmiştir; B3'ün TC/ITDR üstünlüğü için güç  $\approx 1.000$  bulunmuştur. Açık kalan tek anlamsız fark B2 – B1 ITDR (CI sıfırını kapsıyor) — bu bulgunun kendisi sezgisellik sınırlılığının kanıtıdır.
- **Stream / uygulama katmanı eksikti — kısmen giderildi (Bölüm 4.10).** Spec-türevli stream alt-FSM (7 durum, 8 olay, 17 geçerli / 39 geçersiz ikili) modellenmiş ve aynı B1/B2/B3 karşılaştırması uygulanmıştır. Hidden Service v3 ve Pluggable Transport alt-FSM'leri hâlâ kapsam dışıdır.
- **SLR canlı veritabanı erişimi — kısmen giderildi (Bölüm 4.11.2).** OpenAlex ( $\sim 250M$  iş, IEEE/Springer dahil) + CrossRef + arXiv canlı API'leri ile 115 kayıt çekilmiş, 111 tekil referansa indirgenmiştir. Scopus / WoS lisanslı tam-metin + atıf grafi erişimi hâlâ kapsam dışıdır.
- **Latency cross-language ölçümü — kısmen giderildi (Bölüm 4.11.3).**  $\delta$ -lookup + classifyInvalid hot path'inin gcc -O3 C portu eklenmiş, aynı probelerde ortalama  $107\times$  hızlanma ölçülmüştür. Tam Tor C/Rust relay portu hâlâ kapsam dışıdır.

## 6.3 Gelecek Çalışmalar

Bu bölüm üç düzeye ayrılmıştır: (a) bu çalışmanın **metodolojik prototip** seviyesinden **runtime-doğrulamalı güvenlik aracı** seviyesine geçişinin kavramsal çerçevesi (6.3.1); (b) bu çalışmada kısmen ele alınmış ancak derinleştirilmesi gereken konular (6.3.2); (c) Bölüm 6.2.1-B'de "ortam engelleri nedeniyle uygulanmadı" olarak işaretlenen **Shadow + running tor** entegrasyonunun adım adım uygulama planı (6.3.3). Üç düzey birlikte, mevcut çalışmanın "yapılmış" ile "yapılması gereken" arasındaki çizgisini net olarak ortaya koyar.

### 6.3.1 Metodolojik Çalışmadan Gerçek Çalışmaya Geçiş Çerçevesi

Yazılım güvenliği araştırmalarında olgunluk seviyesi tipik olarak NASA'nın Teknoloji Hazırlık Düzeyi (Technology Readiness Level, TRL) ölçeğine benzer bir hiyerarşi ile değerlendirilir. Bu çalışmanın mevcut konumu ve hedef konumu

aşağıdaki tabloda açıkça gösterilmiştir:

| Seviye | Tanım                                        | Bu çalışmadaki karşılığı                                                                                            | Durum                                                              |
|--------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| S1     | Kavramsal model + spec analizi               | $\delta$ tablosu, $\Sigma$ alfabesi, classifyInvalid taksonomisi (Bölüm 3)                                          | <b>Tamamlandı</b>                                                  |
| S2     | Algoritma + simülasyon ile metodoloji ispatı | B1/B2/B3 algoritmaları, N=100 paired karşılaştırma, BCa, exact-t güç analizi (Bölüm 4.1-4.13)                       | <b>Tamamlandı</b>                                                  |
| S3     | Üretim kaynak koduyla yapısal kanıt          | tor 0.4.x statik FSM çıkarımı, 5-durumlu impl FSM, 4 divergence noktası (Bölüm 4.12)                                | <b>Tamamlandı</b>                                                  |
| S4     | Otomaton öğrenme ile pipeline doğrulaması    | Saf-Node L*, 56.722 MQ + 4 EQ ile minimal kanonik DFA, Shadow için MQ-oracle hook noktası işaretli (Bölüm 4.14)     | <b>Tamamlandı</b>                                                  |
| S5     | Simüle edilmiş ağ ortamında runtime trace    | Shadow v3.3.0 + Tor 04.6.10.; 6 relay topoloji; 6.093 olay yakalandı (Bölüm 4.12-B)                                 | <b>Tamamlandı</b>                                                  |
| S6     | Saldırı senaryolarıyla gerçek FPR/TPR ölçümü | Harness-seviyesi replay-attack + circuit-bypass enjeksiyonu; recall = 1.000, precision = 0.369-0.466 (Bölüm 4.12-B) | <b>Kısmen tamamlandı</b> (harness-seviyesi; modifiye binary değil) |
| S7     | Üretim Tor relay'inde alan dağıtımı          | Tor Project ile entegrasyon, gerçek anonim trafikte uzun dönem ölçüm                                                | <b>Yapılmamış</b> (Tor Project işbirliği gerektirir)               |

Mevcut çalışma **S1-S6 seviyelerini tamamlamış veya kısmen tamamlamıştır**. S5 (runtime trace) Shadow simülatörü üzerinde tam olarak gerçekleştirilmiş, S6 (saldırı senaryoları) harness-seviyesi enjeksiyonla kısmen uygulanmıştır. Modifiye tor binary ile hücre-düzeyinde saldırı tetikleme ve L\*'nin running tor'a bağlanması (S6'nın tam hâli) gelecek çalışma olarak kalmaktadır.

**Üç temel teknik dönüşüm.** Metodolojik prototipten gerçek çalışmaya geçiş, koddaki üç noktada yapılacak *oracle değişimi* ile sağlanır; hiçbir mimari yeniden yazım gerekmez:

| Bileşen        | Mevcut (S4 — bu çalışma)                                                                                                    | Hedef (S5-S6 — gerçek çalışma)                                                                                                                |
|----------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Olay kaynağı   | <code>experiments/baselines.mjs</code> içinde algoritmik olay üreticileri (B1 random, B2 greedy, B3 MDT)                    | Shadow içindeki tor instance'larından <code>stem</code> kütüphanesi ile gelen gerçek CIRC event akışı (Adım 6)                                |
| L* MQ oracle   | <code>experiments/g_extensions.mjs</code> içinde statik <code>SUT_DELTA</code> dict üzerine sorgu                           | Aynı dosyadaki <code>MQ()</code> fonksiyonu running tor'a control port probe + circuit state observation (Adım 8)                             |
| FPR/TPR oracle | <code>server/fsm.ts</code> → <code>classifyInvalid</code> yapısal olarak FPR=0 ( $\delta$ -tanımlı geçişleri Invalid demez) | Aynı fonksiyon Shadow-ici modifiye tor client'tan tetiklenen 4 saldırıya karşı koşulu; bağımsız tor log oracle'ı ile FPR/TPR ölçülür (Adım 9) |

Üç değişikliğin **toplam kod ayak izi 500-2000 satır harness'dır**; mevcut algoritma, istatistik ve görselleştirme katmanları *aynen* yeniden kullanılır (Bölüm 4.13 paired N=100 koşusu gerçek olay akışıyla, Bölüm 4.14 L\* yakınsama tablosu running tor ile, Bölüm 4.6-4.11 latency ölçümleri gerçek probe trafiğiyle tekrar edilir). Bu yapısal yeniden

kullanım, çalışmanın **mimari olarak gerçek-çalışma-hazır** olduğunu gösterir; eksik olan üretim değil, koşum ortamıdır.

**Kim, ne kadarda yapar?** Aynı işin üç farklı uygulayıcı profili için gerçekçi süre tahminleri:

| Uygulayıcı                                              | Ortam                                                       | Süre                                           | Maliyet                    |
|---------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------|----------------------------|
| Linux + ağ güvenliği deneyimli araştırmacı              | Kendi laboratuvar makinesi veya kişisel VPS                 | 4-6 hafta tam zamanlı                          | ~5 € (VPS) + zaman         |
| İnsan denetiminde otonom kod ajanı (Claude Code, Devin) | Ajanın kendi cloud VM'i veya yapılandırılmış lokal terminal | 2-3 hafta + ~5-10 saat insan checkpoint review | ~50-500 USD (API/abonelik) |
| Outsourced uzman (Tor / Shadow / ağ güvenliği)          | Sözleşmeli mühendis kendi makinesinde                       | 3-6 hafta                                      | 1.500-4.000 USD            |

Bu üç profil için süreler birbirinden çok uzak değildir; bu, kritik darboğazın *insan zekası ve algoritma kararları değil, ortam erişimi ve simülasyon çalışma süresi* olduğunu gösterir. Mevcut çalışma S1-S4'ün **tek-kişli, tek-oturum, sandbox-kısıtlı bir ortamda tamamlanabilir maksimum** olduğunu ortaya koymaktadır.

### 6.3.2 Konu Bazlı Genişletmeler

- Stream alt-FSM ayrıştırması (kısmen tamamlandı, Bölüm 4.10).** Hidden Service v3 ve Pluggable Transport alt-FSM'leri için aynı  $\delta$ -tablo + classifyInvalid yaklaşımının uygulanması; alt-FSM'lerin hiyerarşik birleştirilmesi.
- Cross-implementation latency.**  $\delta$  tablosunun Rust portu + gerçek Tor relay üzerinde Bölüm 4.11 latency probelarının tekrarı (mevcut C-port karşılaştırması Bölüm 4.11.3'te yapılmıştır; Rust için Bölüm 6.2.1-A'daki gerekçe bkz.).
- İstatistiksel güç genişletmesi.** Bölüm 4.13'teki  $N=100$  paired tasarım, stateCoverage için  $N=193$  gerekliliğini ortaya çıkarmıştır; bu örneklem hedefiyle yeniden koşu ve Hedges  $g$  düzeltmesi.
- Canlı akademik veritabanı entegrasyonu.** Scopus/WoS/IEEE Xplore API anahtarları edinilmesi hâlinde, Bölüm 6.2-v'te belirtilen "açık web ile sınırlı" sınırlılığının kapatılması ve bibliyometrik haritalama (Bölüm 2'nin sayısal güçlendirilmesi).

### 6.3.3 Shadow + Running Tor Entegrasyon Planı (10 adım)

Aşağıdaki plan, Bölüm 6.2.1-B'de listelenen engellerin aşıldığı bir ortamda (Ubuntu 22.04, sudo erişimi, 8 GB+ RAM, kalıcı disk) uygulanır. Toplam tahmini iş yükü 4-6 hafta tam zamanlı bir araştırmacı; otonom kod ajanı (örn. Claude Code, Devin) ile insan denetiminde 2-3 haftaya kadar inebilir. Her adım, mevcut repo'daki bir dosyaya somut bir bağlantı içerir.

| # | Adım            | Teknik içerik                                                                                                                                                                                                             | Repo bağlantısı | Süre      |
|---|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-----------|
| 1 | Ortam hazırlığı | <pre> Ubuntu 22.04 LTS makine (VPS veya lab); apt install cmake gcc g++ libglib2.0-dev libgraph-dev libssl-dev libevent-dev libyaml-dev libelf-dev zlib1g-dev autoconf libtool; sysctl kernel.yama.pttrace_scope=0 </pre> | —               | 1 gün     |
| 2 | Shadow kurulumu | <pre> git clone github.com/shadow/shadow -&gt; ./setup build --jobs 4 &amp;&amp; ./setup install -&gt; shadow --version ile doğrulama (Shadow 3.x bekleniyor) [18] </pre>                                                 | —               | yarım gün |



|    |                                         |                                                                                                                                                                                                                                                                                                                                                   |                                                                           |           |
|----|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-----------|
| 3  | <b>tor binary (debug)</b>               | <code>git clone gitlab.torproject.org/tpo/core/tor , tor-0.4.8 stable branch; ./configure --disable-asciidoc CFLAGS="-g -O0 -fno-omit-frame-pointer"; make -j4</code> → çıktı <code>src/app/tor</code>                                                                                                                                            | Bölüm 4.12 statik FSM ile aynı sürüm                                      | yarım gün |
| 4  | <b>Tor ağ topolojisi</b>                | <code>pip install tornettools</code> ; CollecTor'dan tarihsel consensus + descriptor indirme; <code>tornettools generate --network-scale 0.001</code> → ~30 relay topoloji. Alternatif: Shadow repo <code>examples/tor/minimal/</code> şablonu (5 relay)                                                                                          | —                                                                         | 1 gün     |
| 5  | <b>İlk simülasyon</b>                   | <code>shadow shadow.config.yaml</code> ; 30 dk simülasyon ~30 dk gerçek zaman; çıktı dizini <code>shadow.data/hosts/&lt;relay&gt;/tor.*.stdout</code> + opsiyonel pcap; pipeline'ın uçtan uca koştuğu doğrulanır                                                                                                                                  | —                                                                         | yarım gün |
| 6  | <b>Control protocol harness (Yol A)</b> | Her tor instance'ına <code>ControlPort 9051</code> ekle; Python + <code>stem.control.Controller</code> ile <code>EventType.CIRC</code> dinleyicisi; <code>CIRC.status</code> ( <code>LAUNCHED/EXTENDED/BUILT/FAILED/CLOSED</code> ) → $\Sigma$ alfabesi ( <code>CREATE/EXTEND/READY/FAIL/DESTROY</code> ) eşlemesi; akış HTTP POST ile pipeline'a | <code>server/routes.ts</code> → <code>runFSMSimulation</code>             | 1-2 hafta |
| 7  | <b>Algoritmalar gerçek trafikte</b>     | B1_Random / B2_GreedySC / B3_MDT'yi Shadow olay akışına bağla; aynı metrikleri (stateCoverage, transitionCoverage, ITDR) gerçek ölçümle hesapla; N=100 paired karşılaştırma (Bölüm 4.13) gerçek veriyle tekrarlanır                                                                                                                               | <code>experiments/baselines.mjs</code>                                    | 1 hafta   |
| 8  | <b>L* → running tor</b>                 | MQ oracle fonksiyonunu değiştir; statik <code>SUT_DELTA</code> dict yerine control protocol probe + tor circuit state sorgusu; öğrenilen DFA ↔ Bölüm 4.12 statik FSM karşılaştırması = gerçek divergence kanıtı                                                                                                                                   | <code>experiments/g_extensions.mjs</code> → <code>MQ()</code>             | 2-3 hafta |
| 9  | <b>Saldırı senaryoları</b>              | Shadow içinde modifiye tor client (CREATE atlayan, REPLAY yapan, premature DATA gönderen); 4 saldırı sınıfı (CIRCUIT_BYPASS, REPLAY_ATTACK, HANDSHAKE_SKIP, PREMATURE_DATA) gerçek hücre trafiğinde tetiklenir; <code>classifyInvalid</code> 'in gerçek FPR/TPR'si ölçülür — Bölüm 6.2-ii'nin "yapısal FPR=0" sınırını kıran tek yol              | <code>server/fsm.ts</code> → <code>classifyInvalid</code>                 | 1 hafta   |
| 10 | <b>Tez/paper güncelleme</b>             | Bölüm 4.13'e "real-trace" satırı; Bölüm 4.14'e "L* on running tor" sonucu; Bölüm 6.2.1-B'yi "addressed" olarak işaretler; gerçek FPR/TPR tablosu (yeni Tablo 4.X); kaynakça güncelleme                                                                                                                                                            | <code>thesis/build_thesis.mjs</code> , <code>paper/build_paper.mjs</code> | 3-5 gün   |

**Kritik insan onay noktaları (otonom ajan kullanımı durumunda):** (a) Adım 5 tamamlandığında, ilk Shadow log'ları üretilince "pipeline uçtan uca çalıştı" teyidi; (b) Adım 6'da ilk gerçek CIRC event'inin `classifyInvalid` çıktısı; (c) Adım 9'da saldırı senaryoları kodlanmadan önce, hangi FSM-ihlali türünün "anlamalı saldırı" sayılacağına araştırmacı onayı. Bu üç kontrol noktası, "kod derlendi/çöktü" tipi yüzeysel teyitlerin ötesinde *semantik* doğrulama sağlar.

**Bağımsız doğrulama.** Adım 7-9 sonunda elde edilen rakamlar (gerçek FPR/TPR, gerçek trafikte algoritma performansı, L\*'in tor binary'sinden çıkardığı DFA), bu tezin simülasyon-temelli bulgularını ya teyit eder ya da düzeltir. Her iki sonuç

da bilimsel değer taşır: teyit, modelin dış geçerliğini güçlendirir; düzeltme, Bölüm 4.12'de tespit edilen "spec ↔ implementation divergence"ın niceliksel boyutunu ortaya koyar.

#### 6.3.4 Mevcut Çalışmanın Konumu

Bölüm 4.10-4.14 ve 6.2.1, bu çalışmanın **simülasyon-doğrulamalı metodoloji önerisi + üretim kaynak kodundan yapısal kanıt + algoritma-doğrulamalı  $L^*$  + Shadow runtime doğrulaması** seviyesinde (yukarıdaki S1-S6) olduğunu açıkça konumlandırır. Bölüm 4.12-B'deki Shadow deneyleri, önceki versiyonda "gelecek çalışma" olan S5-S6 adımlarını gerçekleştirmiştir: 9 simülasyondan 6.093 olay yakalanmış, FSM monitörünün recall = 1.000 ile tüm enjekte edilmiş saldırıları tespit ettiği ampirik olarak doğrulanmıştır. Kalan açık adım, modifiye tor binary ile hücre-düzeyinde saldırı tetikleme (S6'nın tam hâli),  $L^*$ 'nin running tor'a bağlanması ve üretim Tor relay'inde alan dağıtımıdır (S7).

## Kaynakça

---

- [1] R. Dingledine, N. Mathewson and P. Syverson, "Tor: The Second-Generation Onion Router," in *Proc. 13th USENIX Security Symposium*, 2004, pp. 303–320.
- [2] S. J. Murdoch and G. Danezis, "Low-Cost Traffic Analysis of Tor," in *Proc. IEEE Symposium on Security and Privacy*, 2005, pp. 183–195.
- [3] A. Johnson, C. Wacek, R. Jansen, M. Sherr and P. Syverson, "Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries," in *Proc. ACM CCS*, 2013, pp. 337–348.
- [4] A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle and T. Engel, "Website Fingerprinting at Internet Scale," in *Proc. NDSS*, 2016.
- [5] I. Karunanayake, N. Ahmed, R. Malaney, R. Islam and S. K. Jha, "De-Anonymisation Attacks on Tor: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2324–2350, 2021.
- [6] M. Backes, A. Kate, P. Manoharan, S. Meiser and E. Mohammadi, "AnoA: A Framework for Analyzing Anonymous Communication Protocols," in *Proc. IEEE CSF*, 2013, pp. 163–178.
- [7] D. Lee and M. Yannakakis, "Principles and Methods of Testing Finite State Machines—A Survey," *Proceedings of the IEEE*, vol. 84, no. 8, pp. 1090–1123, 1996.
- [8] J. Tretmans, "Model Based Testing with Labelled Transition Systems," in *Formal Methods and Testing*, LNCS 4949, Springer, 2008, pp. 1–38.
- [9] J. de Ruiter and E. Poll, "Protocol State Fuzzing of TLS Implementations," in *Proc. 24th USENIX Security Symposium*, 2015, pp. 193–206.
- [10] P. Fiterău-Broștean, R. Janssen and F. Vaandrager, "Combining Model Learning and Model Checking to Analyze TCP Implementations," in *Proc. CAV*, 2016, pp. 454–471.
- [11] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge: Cambridge University Press, 2016.
- [12] D. Dolev and A. C. Yao, "On the Security of Public Key Protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [13] K. Bhargavan, B. Blanchet and N. Kobeissi, "Verified Models and Reference Implementations for the TLS 1.3 Standard Candidate," in *Proc. IEEE S&P*, 2017, pp. 483–502.
- [14] J. Somorovsky, "Systematic Fuzzing and Testing of TLS Libraries," in *Proc. ACM CCS*, 2016, pp. 1492–1504.
- [15] M. Felderer, M. Büchler, M. Johns, A. Brucker, R. Breu and A. Pretschner, "Security Testing: A Survey," *Advances in Computers*, vol. 101, pp. 1–51, 2016.
- [16] A. Pretschner, T. Mouelhi and Y. Le Traon, "Model-Based Tests for Access Control Policies," in *Proc. ICST*, 2008, pp. 338–347.
- [17] B. Kitchenham and S. Charters, "Guidelines for performing Systematic Literature Reviews in Software Engineering," EBSE Technical Report EBSE-2007-01, 2007.
- [18] R. Jansen, K. Bauer, N. Hopper and R. Dingledine, "Methodically Modeling the Tor Network," in *Proc. USENIX CSET*, 2012.
- [19] Tor Project, "Tor Protocol Specification (tor-spec)," 2026. [Online]. Available: <https://spec.torproject.org/tor-spec>. [Accessed: Feb. 2026].
- [20] Microsoft Research, "A Data-Driven FSM Model for Analyzing Security Vulnerabilities," Microsoft Research Technical Report, 2018.

## Ek A — Tam 8 Geçiş Tablosu

Aşağıda Tor devre FSM'inin **25 geçerli geçişi** bire bir listelenmiştir. Bu tablonun dışındaki 105 ikili Invalid kümesini oluşturur ve Ek B'deki sınıflandırıcı ile saldırı vektörlerine eşlenir. Tablo, repo içindeki `server/fsm.ts` dosyasından programatik olarak üretilmiştir; çelişki riski olamayacak şekilde tek kaynak prensibine bağlıdır (single source of truth).

| Mevcut Durum     | Olay ( $\Sigma$ ) | Sonraki Durum    |
|------------------|-------------------|------------------|
| IDLE             | CONNECT           | CONNECTING       |
| CONNECTING       | TLS_OK            | TLS_HANDSHAKE    |
| CONNECTING       | TLS_FAIL          | ERROR            |
| CONNECTING       | TIMEOUT           | ERROR            |
| TLS_HANDSHAKE    | SEND_CREATE       | CREATE_SENT      |
| TLS_HANDSHAKE    | TLS_FAIL          | ERROR            |
| TLS_HANDSHAKE    | TIMEOUT           | ERROR            |
| CREATE_SENT      | RCV_CREATED       | CIRCUIT_BUILDING |
| CREATE_SENT      | TIMEOUT           | ERROR            |
| CIRCUIT_BUILDING | SEND_EXTEND       | CIRCUIT_BUILDING |
| CIRCUIT_BUILDING | RCV_EXTENDED      | CIRCUIT_READY    |
| CIRCUIT_BUILDING | TIMEOUT           | ERROR            |
| CIRCUIT_READY    | SEND_RELAY_DATA   | TRANSMITTING     |
| CIRCUIT_READY    | RCV_RELAY_DATA    | TRANSMITTING     |
| CIRCUIT_READY    | SEND_DESTROY      | CLOSING          |
| CIRCUIT_READY    | RCV_DESTROY       | CLOSING          |
| CIRCUIT_READY    | TIMEOUT           | ERROR            |
| TRANSMITTING     | SEND_RELAY_DATA   | TRANSMITTING     |
| TRANSMITTING     | RCV_RELAY_DATA    | TRANSMITTING     |
| TRANSMITTING     | SEND_DESTROY      | CLOSING          |
| TRANSMITTING     | RCV_DESTROY       | CLOSING          |
| TRANSMITTING     | TIMEOUT           | ERROR            |
| CLOSING          | CIRCUIT_CLOSED    | CLOSED           |
| CLOSING          | TIMEOUT           | CLOSED           |
| ERROR            | CIRCUIT_CLOSED    | CLOSED           |

## Ek B — Saldırı Sınıflandırıcı Envanteri

Aşağıda `classifyInvalid(state, event)` fonksiyonunun 105 geçersiz ikili üzerinde ürettiği etiket dağılımı verilmiştir. Bu tablo da repo içinde programatik olarak hesaplanır; her güncelleme tek satırla yenilenebilir.

| Saldırı tipi       | İkili sayısı | Pay           | LOW | MEDIUM | HIGH | CRITICAL |
|--------------------|--------------|---------------|-----|--------|------|----------|
| GHOST_CIRCUIT      | 38           | 36.2%         | 19  | 15     | 4    | 0        |
| REPLAY_ATTACK      | 25           | 23.8%         | 0   | 14     | 10   | 1        |
| HANDSHAKE_SKIP     | 13           | 12.4%         | 0   | 8      | 5    | 0        |
| CIRCUIT_HIJACK     | 10           | 9.5%          | 0   | 0      | 0    | 10       |
| CREATE_FLOOD       | 8            | 7.6%          | 7   | 1      | 0    | 0        |
| PREMATURE_DATA     | 6            | 5.7%          | 0   | 2      | 4    | 0        |
| CIRCUIT_BYPASS     | 4            | 3.8%          | 0   | 0      | 0    | 4        |
| INVALID_TRANSITION | 1            | 1.0%          | 1   | 0      | 0    | 0        |
| <b>Toplam</b>      | <b>105</b>   | <b>100.0%</b> |     |        |      |          |

Yorumsal not: CIRCUIT\_HIJACK ve CIRCUIT\_BYPASS gibi CRITICAL ağırlıklı vektörler düşük ikili sayısına sahiptir ancak şiddet açısından en yüksek tehdit değerini taşırlar. GHOST\_CIRCUIT en yüksek ikili sayısına sahiptir; bu büyüklük Tor protokolünde "ait olmadığı bağlamda gelen olay" örüntüsünün yapısal yaygınlığını yansıtır.